Continuous Query for Top-K Maximal Sum Intervals over Streaming Data

Zhongshuai Zhang
Beijing Institute of Technology
zhang_zhongshuai@126.com

Xiaochun Yang
Northeastern University
yangxc@mail.neu.edu.cn

Baihua Zheng
Singapore Management University
bhzheng@smu.edu.sg

Rui Zhu
Shenyang Aerospace University
zhurui@sau.edu.cn

Haomin Li
Northeastern University
lihm@mails.neu.edu.cn

Bin Wang
Northeastern University
binwang@mail.neu.edu.cn

ABSTRACT

The continuous identification of top-k maximal sum intervals using a sliding window over a data stream is a critical operation for applications in IoT and beyond. A maximal sum interval is a non-overlapping, contiguous subsequence with the maximal sum in a sequence of real numbers. Existing algorithms are ill-suited for streaming contexts: they either exhaustively enumerate all intervals even for small k values, or depend on indexes that require frequent and costly restructuring. We propose a novel *partition*-based strategy. Our core insight is a partitioning scheme that guarantees that any maximal sum interval is fully contained within a single partition, enabling independent and parallel processing. This design provides two key advantages: it enables early pruning of partitions that cannot contribute to top-k results, drastically narrowing the search space, and it enables efficient, incremental maintenance of the maximal sum intervals in each partition. We develop algorithms for partition construction, incremental partition updates, and partition-based top-k maximal sum interval search. Extensive experiments on real and synthetic datasets demonstrate that our approach significantly improves efficiency.

PVLDB Reference Format:
Zhongshuai Zhang, Xiaochun Yang, Baihua Zheng, Rui Zhu, Haomin Li, and Bin Wang. Continuous Query for Top-K Maximal Sum Intervals over Streaming Data. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Mint-hexagram/C-KMaxI.

1 INTRODUCTION

The Top-k Maximal Sum Intervals (KMaxI) problem [7, 27], a classic problem studied for over 25 years, aims to find the k non-overlapping, contiguous subsequences with the highest sums in a sequence of real numbers. These intervals are ranked such that the i-th interval has the maximum possible sum from the elements disjoint from all higher-ranked (1st to $(i-1)$-th) intervals.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

UNIT: kWh

Sample ID	1	2	3	4	5	6	7	8	9	10	11	···
Generation	69.5	75.2	63.4	76.5	69.4	84.6	62.1	73.7	64.8	72.6	70.4	···
Consumption	67.7	72.8	66.6	72.7	73.4	80.7	68.6	67.9	70.1	67.9	69.2	···
Storage*	1.8	2.4	-3.2	3.8	-4.0	3.9	-6.5	5.8	-5.3	4.7	1.2	···

The 2nd-ranked interval in W_1: $I[1,4]$ *Storage = Generation - Consumption

Figure 1: A running example on power generation, consumption, and storage of a microgrid.

This paper extends the KMaxI problem to the challenging domain of streaming data. We introduce the Continuous KMaxI query (C-KMaxI query in short), which provides real-time insights over a sliding window. While sliding windows can be time- or count-based [6, 10, 14–17, 21, 24, 26, 30], this work focuses on the count-based model. Formally, a C-KMaxI query, denoted as $q(n, s, k)$, continuously monitors the n most recent objects in the query window W. Each time the window advances by s positions, the query returns the top-k maximal sum intervals. The techniques developed, however, are also applicable to time-based sliding windows.

Motivations and Challenges. The C-KMaxI query is motivated by its critical role in real-time analytics across diverse domains. It is essential to identify significant genomic regions in high-throughput bio-informatics sequencing [19, 20, 27, 29], to analyze trends in web scraping [25] and information retrieval [22], and to optimize features for online AI models [18].

A compelling application of C-KMaxI query arises in the Internet of Things (IoT), particularly in real-time monitoring, scheduling, and dispatch in microgrids, which have received significant attention in recent years [12, 35]. Figure 1 illustrates a running example in this context. Here, a C-KMaxI query can continuously analyze a *signed* microgrid signal to identify time periods with the largest *cumulative* energy surplus. Specifically, the input stream can be modeled a surplus/deficit sequence, where evaluate value represents the difference between power generation and consumption. Positive values indicate energy surplus, while negative values indicate energy deficit. The returned top-k non-overlapping maximal-sum intervals therefore correspond to the k most significant, disjoint periods with the highest cumulative surplus. Such intervals are particularly useful for supporting real-time operational decisions, such as energy dispatch and grid balancing. As the system continuously receives new measurements, the query result evolves with the sliding window. For example, the top intervals may change from $I[8, 8]$ with sum 5.8 and $I[1, 4]$ with sum 4.8 in window W_1, to $I[8, 11]$ with sum 6.4 and $I[6, 6]$ with sum 3.9 in window W_2.

This dynamic update capability enables continuous monitoring of emerging surplus periods as the data stream evolves.

Despite its utility, the efficient computation of C-KMaxI queries over data streams faces two formidable challenges. First, the combination of high-velocity data streams and the inherent computational complexity of the C-KMaxI problem makes it difficult to deliver low-latency results, especially for large window sizes. Second, the set of top-k intervals is highly volatile. With each window slide, intervals may split, merge, or be completely replaced. For instance, as shown in Figure 1, when the window advances to W_2, the result interval $I[8, 8]$ merges with adjacent intervals to form a new result interval $I[8, 11]$. This volatility renders static algorithms inefficient, as they cannot update results incrementally and must instead perform costly re-computations from scratch.

The State-Of-The-Art Efforts. Existing research on the KMaxI problem is primarily designed for static data environments and falls into two categories. Scan-based approaches, such as Ruzzo-Tompa [27], identify all maximal intervals through a sequential scan but require a subsequent sorting step to select the top-k, leading to inefficiency especially when k is much smaller than the total count. Index-based approaches, like the Tournament-Tree [7], precompute an index to retrieve the top-k intervals directly without exhaustive comparisons. While efficient in static scenarios, both paradigms are not suited for data streams. Adapting the Tournament-Tree, for instance, would necessitate frequent and costly index reconstructions with every window slide, resulting in prohibitive computational overhead. Consequently, even with modifications, existing algorithms cannot efficiently support the C-KMaxI query.

Our Solution. To overcome these limitations, we propose a novel partition-based framework for efficient C-KMaxI processing. Our core insight is a partitioning strategy that divides the query window W into a set of disjoint partitions $\mathcal{P} = \{P_1, P_2, \cdots, P_p\}$, with a key guarantee: any maximal sum interval is entirely contained within a single partition. This property enables two fundamental advantages: First, the top-k results must come from at most k promising partitions, while all other partitions can be safely pruned, drastically narrowing the search space without relying on a brittle global index. Second, the changes in maximal sum intervals in each partition are intrinsically linked to partition evolution, which can be categorized into four distinct patterns (e.g., unaltered, shrinking, extended, and emergent). We develop efficient update strategies for each pattern, maximizing the reuse of previously identified maximal sum intervals.

Motivated by these benefits, our solution proceeds in three key stages. First, we develop a dynamic programming algorithm (Section 4.1) that constructs all partitions in a single pass over the window. This process simultaneously obtains the top-1 maximal sum interval $I[LM_j, r_j]$ within each partition $P_j = [l_j, r_j]$, forming an initial candidate set.

Next, we introduce a partition-based k-maximal sum interval search algorithm that examines at most k partitions. Partitions are prioritized by the sum of their top-1 maximal intervals $I[LM_j, r_j]$ and processed in descending order. A size-k AVL tree maintains the current top-k intervals. When a partition is visited, a dedicated identification algorithm (Section 4.2.2) finds all maximal sum intervals within it, which are then used to update the AVL tree. Partitions whose top-1 maximal intervals cannot enter the top-k are pruned.

Table 1: Notations

Symbols	Description
o_i	the i-th real number object
$I[i, j]$, $S[i, j]$	the interval from o_i to o_j, the sum of objects in $I[i, j]$
$MaxI[i, j]$	maximal sum interval from o_i to o_j
$q(n, s, k)$	the query, modeled as a sliding window W monitoring most recent n objects, returning the top-k maximal intervals over a data stream $\mathcal{S}$, with s being object update number for each slide
$M[i]$, $M[\cdot]$	the largest sum of an interval starting at position i, $M[\cdot]$ is the collective term for M_i
l_j, r_j, LM_j	the leftmost index in partition P_j, the rightmost index in P_j, the index of the largest M in P_j
pTMS $M[u, v]$	a Potential Target M-Subsequence that has $M[u]/M[v]$ as its strict maximum/minimum M value.

The algorithm terminates after all relevant partitions are processed, and the intervals in the tree form the final results.

Finally, we propose incremental maintenance mechanism to update partitions, their corresponding LM values, and the maximal intervals of promising partitions as the query window slides. We categorize partition evolution into four distinct patterns: unaltered, shrinking, extended, and emergent, each with a specific update logic for its LM position and maximal sum intervals. For instance, in an unaltered partition, both the LM value and all maximal sum intervals remain valid; whereas in an emergent partition, both must be computed from scratch. This systematic treatment enables precise determination of which LM positions and maximal intervals can be reused and which require re-computation, thereby maximizing the reuse of previous results (Section 5).

2 PRELIMINARY

In this section, we first formally define the continuous query for top-k maximal sum intervals over streaming data, and then review several important existing works related to this problem. Table 1 lists important notations used throughout this paper.

2.1 Problem Definition

A *maximal sum interval* (or *maximal interval*, denoted as $MaxI[i, j]$), is a contiguous subsequence $(o_i, \cdots, o_j)$ in a sequence of real numbers $(o_1, o_2, \cdots, o_n)$ whose sum $S[i, j] = \max_{1 \le i \le j \le n} S[i, j]$ is global maximal. In this work, we consider the set of globally ranked, non-overlapping maximal sum intervals, following the iterative definition in [27]. Specifically, the interval with the largest sum in the entire sequence is ranked first; the i-th interval is the maximal-sum interval computed from elements disjoint from all higher-ranked intervals.

To avoid ambiguity under ties, we treat intervals containing non-empty zero-sum prefixes or suffixes as invalid. Otherwise, such intervals could create multiple overlapping intervals with identical sums when adjacent to positive-sum segments. Consequently, if a sequence contains fewer than k valid maximal intervals, a C-KMaxI query may return fewer than k intervals.

Consider the sequence in query window W_1 in Figure 2. The interval $I[10, 10]$ (sum=12) is a global maximum sum interval since no other subsequence has a larger sum. Subsequences like $I[3, 3]$ and $I[11, 13]$ are invalid due to non-positive sums. While $I[1, 2]$ and $I[2, 2]$ both have sum 6, $I[1, 2]$ is invalid as it contains a zero-sum

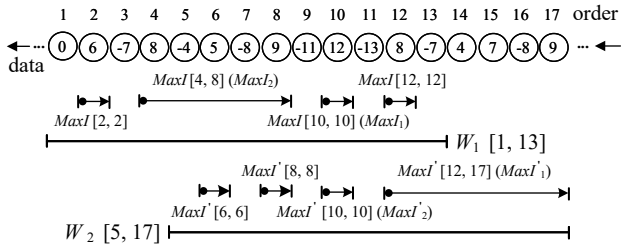

Figure 2: An example of $q(n = 13, s = 4, k = 2)$.

prefix. In addition, each lower rank interval must be disjoint from all higher-ranked ones, e.g., $I[4, 10]$ has a sum larger than $I[4, 8]$ but is inadmissible, as it overlaps with the 1st-ranked interval $I[10, 10]$.

We now formally define **C-KMaxI**, the continuous query for top-k maximal sum intervals over streaming data.

Definition 1. ***C-KMaxI.*** *Given a stream $\mathcal{S}$ of signed values, a C-KMaxI query, denoted as $q(n, s, k)$, maintains a sliding window W of the most recent n objects from $\mathcal{S}$. Each time the window slides by s positions (s objects expire and s new objects arrive), the query returns the top-k maximal intervals $\{MaxI_1, MaxI_2, \ldots, MaxI_k\}$ in the updated window W.*

Figure 2 illustrates an example C-KMaxI query $q(13, 4, 2)$. In window W_1, four maximal intervals exist: $MaxI[2, 2]$, $MaxI[4, 8]$, $MaxI[10, 10]$, and $MaxI[12, 12]$. The top-2 results are $MaxI_1[10, 10]$ (sum = 12) and $MaxI_2[4, 8]$ (sum=10). After the window slides to W_2, the intervals update to $MaxI'[6, 6]$, $MaxI'[8, 8]$, $MaxI'[10, 10]$, and $MaxI'[12, 17]$. The top-2 results become $MaxI'_1[12, 17]$ (sum=13) and $MaxI'_2[10, 10]$ (sum=12).

Streams with signed values naturally arise in many real-world applications. Examples include surplus signals in energy systems (supply–demand), financial price changes, sensor deltas, and residual signals obtained after subtracting a baseline. Residual signals, in particular, convert strictly positive or negative measurements into signed streams (e.g., by subtracting the mean or expected value). In all these scenarios, a C-KMaxI query identifies the top-k intervals with the largest cumulative deviations.

2.2 Related Work - a shorter version

To the best of our knowledge, no existing algorithm efficiently addresses the C-KMaxI problem over streaming data. Prior studies mainly address maximal interval discovery in static sequences or consider different interval optimization objectives.

Maximal interval discovery in static sequences. Finding maximal sum intervals is a classic problem derived from the maximum subarray problem [4]. Existing solutions mainly fall into two categories. Scan-based approaches, such as the Ruzzo–Tompa (RT) algorithm [27], identify all maximal intervals in linear time using a single-pass scan. However, RT always enumerates the full set of maximal intervals, even when only the top-k results are needed, and the intervals are not ranked during enumeration, requiring an additional selection step. Index-based approaches, such as the Tournament-Tree method [7], build a tree-structured index that enables direct retrieval of the highest-ranked interval. After each interval is retrieved, its elements are marked as holes to prevent reuse in subsequent results. While effective for static data, these approaches assume a fixed input sequence and are not designed for continuous updates.

Streaming Interval Extraction. Several studies investigate extracting interval-shaped patterns from data streams under different formulations. For example, [23] proposes an online framework (oPIEC) for detecting probabilistic maximal intervals in noisy streams. This approach operates on probability and identifies threshold-based intervals that may overlap. In contrast, C-KMaxI operates on general signed streams and returns globally ranked non-overlapping maximal intervals, making the problem fundamentally different.

Other Variants. Some work also studies selecting k disjoint subsequences that maximize the total sum. This objective differs fundamentally from C-KMaxI. For instance, given the sequence $(10, -9, 10, -11, 1)$ and $k = 2$, these variants would select intervals $I[1, 1]$ and $I[3, 3]$ (total sum = 20), whereas C-KMaxI selects $MaxI[1, 3]$ and $MaxI[5, 5]$).

Limitations of existing approaches. Although static KMaxI algorithms could in principle be adapted to streaming settings through incremental maintenance, such adaptation remain costly. Scan-based methods must maintain and repeatedly compare all maximal intervals within the sliding window to derive the top-k results, leading to large candidate sets and high update overhead. For index-based methods require continuous structural updates to maintain correctness under window slides, including node insertions, deletions, and restoration of previously excluded elements. These operations introduce significant maintenance costs in high-velocity streams. In contrast, our method avoids both full enumeration and global indexing. By exploiting the partition containment guarantee, we prune partitions that cannot contribute to the top-k results and restrict interval identification and maintenance to a small number of promising partitions, enabling efficient and robust incremental processing over data streams.

2.3 Related Work

To the best of our knowledge, no existing algorithm efficiently addresses the C-KMaxI problem over streaming data. For static sequences, finding all (or top-k) maximal intervals is a classic problem originating from the maximum subarray problem [4]. Existing solutions mainly fall into two categories: *scan-based approaches*, such as the Ruzzo-Tompa algorithm [27], and *index-based approaches*, like the Tournament-Tree algorithm [7]. Numerous related variants have also been studied [8, 11, 13].

Ruzzo-Tompa (RT) algorithm. RT finds all maximal intervals in a single pass. It maintains a list of candidate subsequences, treating each new positive-valued object as a new length-one candidate that can be merged with the most recent candidate. A pointer-based mechanism efficiently checks mergeability, enabling the algorithm to run in $O(n)$ time. However, RT always enumerates all maximal intervals, even when only the top-k are required. Since these intervals are not ranked, an additional selection step is need to determine the top-k.

Tournament-Tree algorithm. This algorithm addresses the above inefficiency by maintaining a tree-structured index in which the root always represents the global maximum subsequence, enabling direct retrieval of the top-ranked maximal interval. Once an interval is selected as a result, its elements are marked as "hole" and

excluded from subsequent retrievals. However, this index structure is designed for static data and is unsuitable for streaming setting: sliding window updates require frequent node insertions, deletions, and rebalancing.

Streaming/Online Interval Extraction. Several studies have explored extracting interval-shaped patterns from data streams under different formulations. For example, [23] proposes an online framework (oPIEC) for recognizing events in noisy streams by computing probabilistic maximal intervals. This approach transforms the task into detecting maximal non-negative sum intervals on a derived probability sequence. However, it differs from C-KMaxI in both input semantics and output objective: it operates on probability streams and returns threshold-based maximal intervals that may overlap.

Other Variants. Related work also considers selecting k disjoint subsequences that maximize the total sum. This objective differs fundamentally from C-KMaxI. For instance, given the sequence $(10, -9, 10, -11, 1)$ and $k = 2$, these variants would select intervals $I[1,1]$ and $I[3,3]$ (total sum = 20), whereas C-KMaxI selects $MaxI[1,3]$ and $MaxI[5,5]$).

Discussion. In principle, static KMaxI algorithms could be adapted to streaming settings via incremental maintenance, but such adaptation remain significantly more expensive than our approach. For scan-based methods such as Ruzzo-Tompa (RT), incremental maintenance can avoid recomputing intervals from scratch after each window slide. However, without a mechanism to restrict the search space, the algorithm must still maintain and repeatedly compare *all* maximal intervals in the window to determine the top-k, resulting in large candidate sets and high update costs.

For index-based methods such as the Tournament-Tree, incremental updates may delete expired nodes and insert new ones. Nevertheless, sliding windows introduce intrinsic structural overhead: the tree must be continuously rebalanced and updated to maintain correct retrieval semantics, and previously marked "holes" must be restored when affects elements expire. These maintenance costs can dominate in high-velocity streams. In contrast, our method neither enumerates all maximal intervals nor relies on a global static index. By exploiting the partition containment guarantee, we can prune partitions that cannot contribute to the top-k results and restrict interval identification and maintenance to a small number of promising partitions, enabling efficient and robust incremental processing over data streams.

3 PARTITIONING

This section formalizes our partitioning strategy for optimizing C-KMaxI queries. The core of this approach is a guarantee that any maximal interval resides entirely within a single partition, enabling independent processing. This property yields two key characteristics, as established earlier: the early pruning of non-result-bearing partitions and efficient incremental maintenance via partition evolution patterns. We begin by formally defining partitions and will subsequently demonstrate how these characteristics are realized.

Definition 2. ***Partition.*** *A partition $\mathcal{P} = \{P_1, P_2, \ldots, P_p\}$ of a query window $W = (o_1, o_2, \ldots, o_n)$ is a division of W into contiguous, non-overlapping segments that collectively cover W. Each partition $P_i = \{o_{l_i}, o_{l_i+1}, \ldots, o_{r_i}\} = [l_i, r_i]$ with $1 \le l_i \le r_i \le n$ has a right boundary at position r_i, which is defined as the position that maximizes the cumulative sum from the start of the partition P_i; that is, $S[l_i, r_i] \ge S[l_i, t]$ for all $t \in [l_i, n]$.*

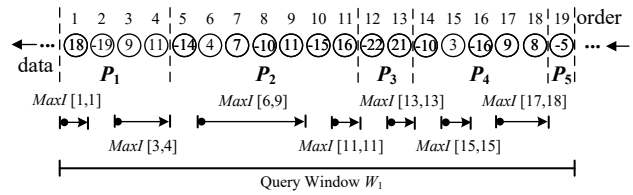

Figure 3: An example of partitions.

Figure 3 illustrates an example of partitions for $W_1 = (o_1, \cdots, o_{19})$. Objects are divided into five partitions: P_1, P_2, P_3, P_4, and P_5. Consider partition $P_1 = (o_1, o_2, o_3, o_4) = [1, 4]$. Its right boundary is at position 4 (i.e., $r_1 = 4$). This means that for a subsequence starting from 1, the cumulative sum is maximized at position 4; any interval $[1, j]$ with $j \neq 4$ yields a smaller cumulative sum. This property extends to any subsequence starting from $s \in [1, 4]$, as formally established in Lemma 1. This follows directly from the functional definition of the partition's right boundary. We omit the formal proof in the interest of brevity.

Lemma 1. ***Right-bound Alignment.*** *Let partition $P_i = (o_{l_i}, o_{l_i+1}, \ldots, o_{r_i}) = P[l_i, r_i]$ be a partition of size greater than 1 of $W = (o_1, o_2, \cdots, o_n)$, where r_i is its right boundary as defined in Definition 2. Then, for every start index $j \in [l_i, r_i]$, the cumulative sum starting from o_j also attains its maximum at the same right boundary r_i. Formally, $\forall j \in [l_i, r_i]$, $S[j, r_i] = \max_{t \in [j,n]} S[j, t]$.*

Beyond the right-bound alignment property, our partitioning scheme exhibits a more profound property: any maximal interval is entirely contained within a single partition. This guarantee is formally encapsulated by Theorem 1.

Theorem 1. *No maximal interval overlaps with any two adjacent partitions.*

Proof. We proceed by contradiction. Assume the theorem is false, i.e., meaning there exists a maximal interval $MaxI[a, b]$ that overlaps with two adjacent partitions $P_i = [l_i, r_i]$ and $P_{i+1} = [l_{i+1}, r_{i+1}]$ where $r_i = l_{i+1} - 1$, $l_i \le a \le r_i$, and $b \ge l_{i+1}$. By the definition of a maximal interval, we have $S[a, b] > S[a, r_i]$. On the other hand, the right-bound alignment property of a partition guarantees that $S[a, b] \le S[a, r_i]$. These two constraints introduce a contradiction, implying that our initial assumption is false, thereby proving the theorem. □

For W_1 in Figure 3, there are in total seven maximal intervals: $\{MaxI[1,1], MaxI[3,4], MaxI[6,9], MaxI[11,11], MaxI[13,13], MaxI[15,15], MaxI[17,18]\}$. As guaranteed by Theorem 1, each maximal interval is contained entirely within a single partition (e.g. $MaxI[1,1]$ and $MaxI[3,4]$ lie in partition P_1).

A key benefit of our partitioning strategy is its support for efficient query processing by focusing on local maximal intervals. Since Theorem 1 guarantees that any maximal interval is entirely contained within a single partition, partitions can be processed independently. This containment enables a crucial pruning heuristic: we first identify the top-1 maximal interval within each partition. If this local champion for a given partition P_i does not qualify for

the global top-k list, then no other intervals within P_i can be query results. Therefore, the partition P_i can be safely pruned.

Consider the query in Figure 3 that searches for top-3 maximal intervals within W_1. The top-1 maximal intervals with each partition are $MaxI[3,4]$ (sum 20) in P_1, $MaxI[11,11]$ (sum 16) in P_2, $MaxI[13,13]$ (sum 21) in P_3, and $MaxI[17,18]$ (sum 17) in P_4. P_5, only containing a non-positive object, can be skipped as it does not contain any maximal intervals. Among these, $MaxI[11,11]$ from P_2 is not a top-3 candidate. Consequently, no other interval in P_2, such as $MaxI[6,9]$, can be a query result and the entire partition can be pruned.

4 THE INITIALIZATION CONSTRUCTION

In this section, we first introduce the partition construction algorithm and then propose a search algorithm to locate k-maximal intervals based on partitions.

4.1 The Partition Construction

We propose a dynamic programming-based partition construction algorithm. Moreover, while constructing the partitions, it provides direct access to the top-1 maximal interval within each partition, without requiring extra computation.

Given a query window $W = (o_1, \cdots, o_n)$, we define two arrays $M[1 \cdots n]$ and $E[1 \cdots n]$. Here, $M[i]$ stores the largest cumulative sum of any contiguous interval starting at position i, and $E[i]$ records the ending position of the corresponding interval, i.e., $S[i, E[i]] = M[i]$. Formally,

$$M[i] = S[i, E[i]] = \max_{1 \le i \le j \le n} \sum\nolimits_{\mu=i}^{j} o_\mu$$

Both arrays can be computed in reverse order using the following recurrence relation:

$$(M[i], E[i]) = \begin{cases} (o_i + M[i+1], E[i+1]), & \text{if } M[i+1] > 0, \\ (o_i, i), & \text{if } M[i+1] \le 0 \end{cases} \quad (1)$$

with the base case $M[n] = o_n$ and $E[n] = n$. Intuitively, if $M[i+1] \le 0$, the single-object interval $I[i,i]$ (i.e, just o_i) yields the maximal sum starting at i, as including any subsequent interval would only decrease the sum. Crucially, any position i where $M[i] \le 0$ is identified as a *cutting point*, as formalized in Definition 3. A non-positive $M[i]$ indicates that no interval $I[j, \cdot]$ starting at position $j < i$ can achieve a larger sum by extending to include any interval starting at position i. Consequently, these positions naturally form the boundaries of our partitions.

Definition 3. ***Cutting Points.*** *A cutting point is a position i in the query window W for which the value $M[i] \le 0$. The object o_i located at a cutting point is the physical marker for that boundary.*

A partition is determined by two consecutive cutting points, inclusive of its left cutting point and exclusive of the right one. The window boundaries (i.e., positions 1 and n of a query window of size n) serve as implicit partition boundaries if they are not themselves cutting points. As illustrated in Figure 4, the cutting points o_5, o_{12}, o_{14}, and o_{19} divide the entire sequence W_1 into five partitions P_1, P_2, P_3, P_4, and P_5.

Algorithm 1: Partition Construction

Input: The object array $o[1, \cdots, n]$
Output: M values; partitions $\mathcal{P}$; indices of the largest M in each partition $\mathcal{LM}$, and set of cutting points C.

```
Initialize array M[1, ···, n], partition set P, the largest M index set LM,
  the cutting point index set C, the left/right cutting point index lc/rc;
M[n] ← o[n], rc ← n + 1;
for i ← n − 1 to 1 do
    if M[i + 1] ≤ 0 then // scan a cutting point
        C.add (i + 1); lc ← i + 1, M[i] ← o[i] ;
        if P[lc, rc − 1] is a degenerate partition then
            rc ← lc, LM ← i ;
        else // P[lc, rc − 1] is a non-degenerate partition
            P.add (P[lc, rc − 1]), LM.add (LM), rc ← lc, LM ← i;
    else // enter a non-degenerate partition
        M[i] ← o[i] + M[i + 1];
        if M[i] > M[LM] then
            LM ← i;
if P[1, rc − 1] is a non-degenerate partition then
    P.add (P[1, rc − 1]), LM.add (LM);
return M, P, LM, C;
```

The partition construction process is detailed in Algorithm 1. The algorithm performs a reverse scan of the window W, computing $M[i]$ values and identifying cutting points (positions i where $M[i] \le 0$). It maintains two pointers, lc and rc, to track the boundaries of the next partition to be formed. When a new cutting point is found, objects within the segment $[lc, rc-1]$ are finalized as a new partition, and lc is updated to rc. Here, we want to highlight that not all partitions are retained in our algorithm; a degenerate partition containing only a single non-positive object is discarded, as it cannot contain any positive-sum maximal intervals. Only non-degenerate partitions are returned to serve C-KMaxI query.

The algorithm also maintains a list $\mathcal{LM}$ that stores, for each non-degenerate partition, the indices of the largest $M[i]$ value. Formally, for a non-degenerate partition $P_i = [l_i, r_i]$, $LM_i = \arg\max_{j \in [l_i, r_i]} M[j]$. Accordingly, the interval $I[LM[i], r_i]$ is the top-1 maximal interval in P_i, which will be formally presented in Lemma 4. This provides direct access to the top-1 maximal interval in each partition, facilitating the processing of C-KMaxI as detailed later.

In addition, the algorithm uses a list C to store the indices of all cutting points. These cutting points help determine how partitions evolve after the query window slides, as detailed in Section 5.1. Finally, we emphasize that the partitions generated by this process adhere strictly to the definition given in Definition 2. Specifically, as established by Lemma 2, the right boundary of each partition $P = [l_i, r_i]$ is correctly positioned at r_i, i.e., $\forall j \in [l_i, r_i]$, $E[j] = r_i$. The proof is omitted, as it can be derived directly from the definition of cutting points. This property is precisely why Algorithm 1 does not need to maintain $E[\cdot]$ array explicitly.

The example in Figure 4 demonstrates this process. The right boundary rc is initialized to 20, making position 19 the initial right boundary of the initial scanned partition. Since $M[19] < 0$, the segment $[19, 20)$ is identified as a degenerate partition (containing only o_{19}) and is discarded. The algorithm then processes a non-degenerate partition, where LM is updated to 17 (as $M[17]$ is the maximum in this partition). Upon encountering the next cutting point at position 14, a non-degenerate partition $P_4 = [14, 19)$ is recorded. Partitions P_3 and P_2 are identified similarly. Finally, the

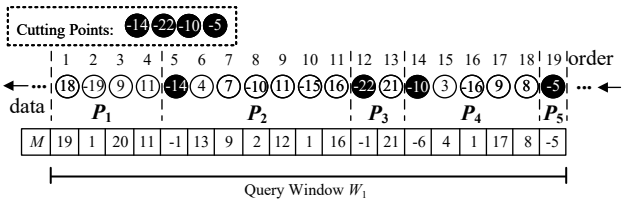

Figure 4: Constructing partitions.

leftmost partition $P_1 = [1, 5)$ is verified as non-degenerate and recorded. In partition $P[1] = [1, 4]$, interval $I[2, 4]$ is the interval I_2 that achieves the cumulative sum of $M[2] = 1$, and interval $I[3, 4] = I_3$ that achieves a cumulative sum of $M[3] = 20$. Accordingly, $LM_1 = 3$, and $I[3, 4]$ is the top-1 interval within partition P_1.

Lemma 2. *In a non-degenerate partition $P_i = [l_i, r_i]$ constructed by Algorithm 1, r_i is P_i's right boundary, i.e., $\forall j \in [l_i, r_i], E[j] = r_i$.*

4.2 Partition-based k-Maximal Interval Search

In a streaming context, the query window evolves continuously. A naive method would compute all maximal intervals for each new window, which is computationally prohibitive. Our key insight is that the top-k results in any window originate from at most k partitions. This is because each result interval is contained within a single partition, and k disjoint result intervals can belong to at most k partitions (or all partitions if the total number of partitions is smaller). In practice, the number of partitions is typically much larger than k, making this observation particularly useful for pruning the search space. We therefore rely on the top-1 maximal interval within each partition to identify the k most promising partitions.

4.2.1 Search algorithm. Instead of generating all candidates upfront, we fully exploit identified partitions to focus only on promising partitions (at most k) and retain the identified intervals to form a candidate set $\mathcal{CM}$ for incremental maintenance in subsequent windows.

As stated earlier, for any non-degenerate partition $P_j = [l_j, r_j]$, the interval $I[LM_j, r_j]$ is the top-1 maximal interval in P_j. This is formally presented in Lemma 4. To facilitate the proof of this lemma, we introduce the maximality of the complete set of maximal intervals $\mathcal{M}_\mathcal{I} = \{MaxI[u_1, v_1], \ldots, MaxI[u_{n'}, v_{n'}]\}$. This set contains only maximal intervals, and it is complete, i.e., no positive-sum interval can be formed from elements disjoint from all intervals in $\mathcal{M}_\mathcal{I}$. We now show that any positive-sum interval $I[i, j]$ not in $\mathcal{M}_\mathcal{I}$ must have overlaps with a maximal interval $MaxI[u_m, v_m]$ in $\mathcal{M}_\mathcal{I}$ with $S[i, j] \leq S[u_m, v_m]$, formally stated in Lemma 3.

Lemma 3. *$\forall\, I[i, j] \notin \mathcal{M}_\mathcal{I}$ with $S[i, j] > 0$, $\exists\, MaxI[u_m, v_m] \in \mathcal{M}_\mathcal{I}$ with $MaxI[u_m, v_m] \cap I[i, j] \neq \emptyset$ such that $S[u_m, v_m] \geq S[i, j]$.*

Proof. Since $\mathcal{M}_\mathcal{I}$ is complete, a positive-sum interval $I[i, j]$ must overlap with at least one maximal interval. Assume that the lemma is false. Then, there exists a positive-sum interval $I[i, j] \notin \mathcal{M}_\mathcal{I}$ such that for every maximal interval $MaxI[u_m, v_m] \in \mathcal{M}_\mathcal{I}$ that overlaps with $I[i, j]$, we have $S[u_m, v_m] < S[i, j]$.

Now, consider the iterative process that selects the 1st, 2nd, $\cdots$, maximal interval. Let $MaxI[u_m, v_m]$ be the first maximal interval in this process that overlaps with $I[i, j]$. By the selection rule, this interval has the largest possible sum among all intervals disjoint from all previously selected ones. However, since $I[i, j]$ is disjoin from all intervals selected before $MaxI[u_m, v_m]$ (otherwise $MaxI[u_m, v_m]$ would NOT be the first to overlap with it), and given our assumption that $S[u_m, v_m] < S[i, j]$, the selection process should have chosen $I[i, j]$ (or an interval containing it) over $MaxI[u_m, v_m]$ for the current step, as $I[i, j]$ offers a larger sum. Once $I[i, j]$ (or an interval containing it) is selected as a maximal interval, $I[u_m, v_m]$ will not become a maximal interval due to the fact that $I[u_m, v_m]$ overlaps with $I[i, j]$. This contracts the fact that $I[u_m, v_m] \in \mathcal{M}_I$. Therefore, our initial assumption must be false, and our proof complete. there must exist at least one overlapping maximal interval $MaxI[u_m, v_m]$ with $S[u_m, v_m] \geq S[i, j]$. □

Algorithm 2: Partition-based k-Maximal Interval Search

Input: Partitions $\mathcal{P} = \{P_1, P_2 \cdots, P_p\}$ with their respective top-1 M values $(M[LM_1], M[LM_2], \cdots, M[LM_p])$, the candidate maximal interval set $\mathcal{CM} = \{CM_1, CM_2, \cdots, CM_p\}$, required k

Output: The top-k maximal intervals, the candidate maximal interval set.

```
Initialize an AVL tree avl with the size of k;
insert all M[LM_j] for j ∈ [1, p] into avl;
while find M[LM_i] in avl in descending order do
    incrementally update CM_i; // Algorithm 5
    foreach maximal interval MaxI[u, v] in CM_i do
        insert the sum of MaxI[u, v] into avl;
return maximal intervals whose sums are in avl, CM;
```

Lemma 4. *For a partition P_j that ends at position r_j, the interval $I[LM_j, r_j]$ is the maximal interval with the largest sum within P_j.*

Proof. First, we prove that $I[LM_j, r_j]$ has the largest sum in $P_j = [l_j, r_j]$. This can be directly derived based on the definition of LM_j and Lemma 2, i.e., $S[LM_j, r_j] = \max_{i=l_j}^{r_j} S[i, r_j]$.

Next, we prove $I[LM_j, r_j]$ is a maximal interval via contradiction. Assume interval $I[LM_j, r_j]$ is not a maximal interval. According to maximal intervals' maximality established in Lemma 3, there exists a maximal interval $MaxI[u, v]$, such that $MaxI[u, v] \cap I[LM_j, r_j] \neq \emptyset$ and $S[u, v] \geq S[LM_j, r_j]$. As guaranteed by Theorem 1 and the fact that $MaxI[u, v] \cap I[LM_j, r_j] \neq \emptyset$, maximal interval $MaxI[u, v]$ must be located entirely within one partition (i.e., partition P_j), i.e., $l_j \leq u \leq v \leq r_j$. From $S[u, v] \geq S[LM_j, r_j]$, we have $S[u, v] = M[u] - M[v + 1] = S[u, r_j] - S[v + 1, r_j] \geq S[LM_j, r_j] = M[LM_j]$. In other words, $M[u] \geq M[LM_j] + M[v + 1]$. The definition of M ensures that $M[v + 1] > 0$, so $M[u] > M[LM_j]$, which conflicts that $M[LM_j]$ has the largest cumulative sum than any other $M[i]$ for $\forall i \neq LM_j \in [l_j, r_j]$. Consequently, our assumption is invalid and $I[LM_j, r_j]$ is a maximal interval. □

Algorithm 2 details the "Partition-based k-Maximal Interval Search" approach. It utilizes a size-k AVL tree to dynamically track the current top-k maximal intervals. Partitions P_j whose local champion (e.g., the interval $I[LM_j, r_j]$) is inserted into this tree are deemed promising. The algorithm processes these promising partitions (up to k) in descending order of their champion's sum. For each such partition P_i, CM_i stores all maximal intervals identified within it. The union of all such CM_i forms the final candidate maximal interval set $\mathcal{CM}$.

4.2.2 Identifying maximal intervals in a partition. The operation of locating maximal intervals within partition P_j that are disjoint from

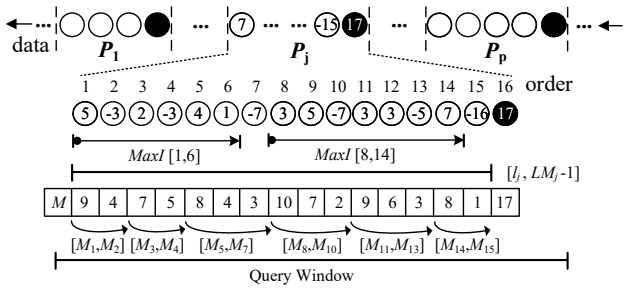

Figure 5: An example of maximal intervals in range $[l_j, LM_j - 1]$ in P_j where $l_j = 1$ and $LM_j = 16$.

the top-1 interval $I[LM_j, r_j]$ is fundamental to our partition-based search algorithm (i.e., a core operation in Algorithm 5). To facilitate this, we first establish a necessary but not sufficient condition that all maximal intervals must satisfy, as formalized in Lemma 5.

Lemma 5. *Let $MaxI[u, v-1]$ be a maximal interval within the sub-range of $[l_j, LM_j - 1]$ of partition $P_j = [l_j, r_j]$. Then, for any position $i \in (u, v]$, $M[u] > M[i]$; for any position $i \in [u, v)$, $M[i] > M[v]$. That is, within a maximal interval, the start position u is a strict local maximum and the position v immediately following the end position is a strict local minimum of the $M[\cdot]$ values.*

Proof. Let $MaxI[u, v-1]$ be a maximal interval. We prove the two properties by contradiction. *1. Proof that $M[u] > M[i]$ for $i \in (u, v]$:* Assume there exists $i \in (u, v]$ such that $M[u] \leq M[i]$. Based on Lemma 2: $M[u] = S[u, r_j]$ and $M[i] = S[i, r_j]$. Since $M[u] \leq M[i]$, we have $S[u, r_j] \leq S[i, r_j]$ and hence $S[u, v-1] \leq S[i, v-1]$. Furthermore, $I[u, v-1]$ and $I[i, v-1]$ overlap on $[i, v-1]$. This contradicts the fact that $I[u, v-1]$ is a maximal interval. *2. Proof that $M[i] > M[v]$ for $i \in [u, v)$:* Assume there exists $i \in [u, v)$ such that $M[i] \leq M[v]$. Consider the interval $I[u, i-1]$. We have $S[u, i-1] = M[u] - M[i]$. Since $M[i] \leq M[v]$, $S[u, i-1] \geq M[u] - M[v] = S[u, v-1]$. Again, $I[u, v-1]$ overlap on $[u, i-1]$. This contradicts the fact that $I[u, v-1]$ is a maximal interval.

As both cases lead to contradictions, the lemma is proven. □

Lemma 5 states that $I[u, v-1]$ can be a maximal interval only if the corresponding M-subsequence $M[u, v]$ has $M[u]$ as its strict maximum and $M[v]$ as its strict minimum. Any M-subsequence satisfying this condition is defined as a *Potential Target M-Subsequence* (pTMS for short). However, not every pTMS corresponds to a maximal interval. For instance, in Figure 5, $M[5, 7]$ is a pTMS, yet $I[5, 6]$ is not a maximal interval. Thus, while pTMS provides a filtered candidate set, further refinement is needed.

Our objective is to identify a *Target M-Subsequence* (TMS for short), $M[u, v]$, for which the corresponding interval $I[u, v-1]$ is indeed a maximal interval. The key to this refinement is the mergeability of pTMS: two pTMS, $M[u_1, v_1]$ and $M[u_2, v_2]$ (with $v_1 < u_2$) are mergeable if their concatenation $M[u_1, v_2]$ also constitutes a pTMS. We then establish the central result that a pTMS is a true TMS if and only if it is non-mergeable in Lemma 6.

Definition 4. ***Mergeability of two pTMS.*** *Let $M[u_1, v_1]$ and $M[u_2, v_2]$ be two pTMS in a partition $P_j = [l_j, r_j]$ such that $l_j \leq u_1 < v_1 < u_2 < v_2 < LM_j$. They are mergeable iff the concatenated sequence $M[u_1, v_2]$ itself constitutes a valid pTMS (i.e., $M[u_1]$ is the strict maximum and $M[v_2]$ is the strict minimum over the extended range $[u_1, v_2]$). If this condition does not hold, the two pTMS are non-mergeable.*

Lemma 6. *A pTMS within $[l_j, LM_j - 1]$ is a TMS iff it is non-mergeable.*

Proof. We prove the equivalent contrapositive statement: if a pTMS is mergable, then it is NOT a TMS. Let $M[u_1, v_1]$ be a pTMS that is mergeable with another pTMS $M[u_2, v_2]$ where $l_j \leq u_1 < v_1 < u_2 < v_2 \leq LM_j - 1$. This implies: 1) $M[u_1]$ is the strict maximum in $[u_1, v_2]$, and 2) $M[v_2]$ is the strict minimum in $[u_1, v_2]$.

Now, consider the interval $I[u_1, v_2 - 1]$. We have $S[u_1, v_2 - 1] = M[u_1] - M[v_2]$. As $M[v_2]$ is the strict minimum in $[u_1, v_2]$, $S[u_1, v_2 - 1] > M[u_1] - M[v_1] = S[u_1, v_1 - 1]$. The intervals $I[u_1, v_2 - 1]$ and $I[u_1, v_1 - 1]$ overlap on $[u_1, v_1 - 1]$. According to Lemma 2, $I[u_1, v_1 - 1]$ cannot be a maximal interval because it is overlapped by another interval $I[u_1, v_2 - 1]$ with a strictly larger sum. Therefore, $M[u_1, v_1]$ is not a TMS. Our proof completes. □

Inspired by Lemma 6, we need to identify non-mergeable pTMS. Directly determining non-mergeability during a scan requires examining all future M values, leading to repeated scans. To avoid this inefficiency, we introduce *Maximal Monotonically Decreasing M-SubSequence* (MDMS for short), the longest contiguous subsequences where M values directly decrease (e.g., $M[1, 2]$ and $M[3, 4]$ in Figure 5). MDMS are pivotal because they i) are pTMS, ii) can be identified in a single scan, and iii) can be iteratively merged according to Definition 4 to ultimately form TMS.

Our algorithm, outlined in Algorithm 3, proceeds as follows. We scan the M sequence in each partition P_i in order (from l_i to $LM_i - 1$), identifying MDMS on the fly. We maintain an ordered list L of current pTMS candidates. Each newly identified MDMS appends to the tail of L. We then check its mergeability with existing pTMS in L, scanning sequentially from the tail (immediately preceding the new MDMS) towards the head. If a merge is possible, we merge them, delete all pTMS contained within the new merged range in L, and re-check the mergeability of the new, longer pTMS with the remaining elements in L. Once this local merge process is complete, the tail pTMS (originating from the identified MDMS) is guaranteed to be non-mergeable with any other pTMS in L. We then continue the scan to identify the next MDMS and repeat the process. This process continues until all MDMS are processed, with the final list L containing the desired TMSs.

Figure 6 shows this process when MDMS $M[5, 7]$ is scanned (Step ②). The list contains $M[1, 2]$ and $M[3, 4]$. The algorithm checks its mergeability with these candidates from tail to head sequentially. While $M[5, 7]$ cannot merge with $M[3, 4]$, it is mergeable with $M[1, 2]$, forming the new, longer pTMS $M[1, 7]$. This new pTMS now fully encompasses the range of $M[3, 4]$, making it obsolete. Consequently, the list is updated to $M[1, 7]$.

A naive implementation that checks each new MDMS against every pTMS in L is inefficient, as it performs many unnecessary checks. We optimize this with a *skip-pointer* mechanism, based on two key insights: let $M[u_1, v_1]$ and $M[u_2, v_2]$ ($v_1 < u_2$) be two pTMSs, 1) if $M[v_1] \leq M[v_2]$, $M[u_2, v_2]$ *cannot* merge with $M[u_1, v_1]$ and any pTMS located before it in L, as this would violate the requirement that the last element of any pTMS must have the

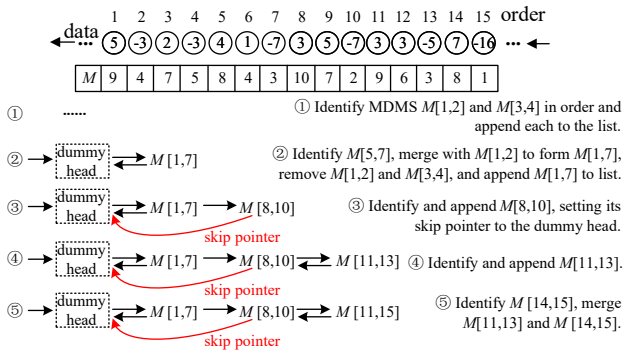

Figure 6: Build a skip-pointer when the algorithm scans $M[8, 10]$.

Algorithm 3: Identify Maximal Intervals

Input: Array $M[l_j \ldots LM_j - 1]$
Output: Maximal intervals

1 Initialize an empty double linked list L with a dummy head;
2 **while** *traverse the array* M **do**
3 scan a Maximal Monotonically Decreasing M SubSequence $M[i, j]$;
4 $M[u, v] \leftarrow L.tail$; $L.append(M[i, j])$;
5 **while** $M[u, v] \neq NULL$ **do** // `traverse` L reversely
6 **if** $M[u] > M[i] \wedge M[v] > M[j]$ // `mergeable` **then**
7 $ptr \leftarrow M[u, v].prev$; L.delete($M[u, v]$);
8 $L.M[i, j] \leftarrow M[i, j] \leftarrow M[u, j]$; $M[u, v] \leftarrow L.ptr$;
9 **else if** $M[v] \leq M[j]$ **then**
10 $L.M[i, j].prev \leftarrow M[u, v]$;
11 break; // `stop traversing` L
12 **else** // $M[u] \leq M[i] \wedge M[v] > M[j]$
13 $L.M[i, j].prev \leftarrow M[u, v].prev$;
14 $M[u, v] \leftarrow M[u, v].prev$;
15 **return** elements of L;

strict minimum M value; 2) in the case of not satisfying 1), i.e., $M[v_1] > M[v_2]$, if $M[u_1] \leq M[u_2]$, $M[u_1, v_1]$ *cannot* merge with $M[u_2, v_2]$ and any pTMS located after it in L, as this would violate the requirement that the first element of any pTMS must have the strict maximum M value.

We implement both insights by re-purposing the `prev` pointer in the ordered list L. For any pTMS in L, its `prev` pointer does not simply point to the immediate predecessor. Instead, it points to the nearest preceding pTMS in L with a smaller or equal last M value, or to the dummy head element if no such preceding pTMS exists. This pointer effectively creates a "skip list" that allows the algorithm to bypass directly all intervening pTMS that are guaranteed to be unmergeable due to their smaller or equal initial M values and larger last M values.

The mergeability of a pTMS (a new MDMS $M[i, j]$ or one formed by a previous merge) and an existing pTMS $M[u, v]$ in L is determined by evaluating the extended range $M[u, j]$. This leads to three cases based on whether $M[u]$ is the strict maximum and $M[j]$ is the strict minimum in the extended range $M[u, j]$.

- Case i): both conditions hold (Lines 6-8). This sequence is merged into a new pTMS $M[u, j]$. All pTMS contained within this new range are removed from L, and the algorithm continues checking the new pTMS against the remaining elements in L.
- Case ii): $M[j]$ is not minimal (Lines 9-11). Neither $M[i, j]$ can merge with $M[u, v]$ nor it can bypass $M[u, v]$, because a future pTMS $M[i', j']$ identified after $M[i, j]$ may has the last element $M[j']$ no less than $M[v]$. $M[i', j']$ is not allowed to bypass $M[u, v]$ to verify whether it merges with pTMS located before $M[u, v]$ in L, so the `prev` pointer of $M[i, j]$ is set to $M[u, v]$. Then the verification is terminated. Note that the `prev` pointer becomes the skip-pointer if $M[u, v]$ and $M[i, j]$ are not adjacent ($M[u, v]$ is not the last pTMS in L).
- Case iii): only $M[j]$ is minimal (Lines 12-14). Not only $M[u, v]$ can not merge with $M[i, j]$ and its future pTMS, but also it can not prevent these pTMSs from merging with ones located before $M[u, v]$ in L. $M[u, v]$ can be bypassed, so the `prev` pointer of $M[i, j]$ is set to that of $M[u, v]$, and the algorithm continues checking the $M[i, j]$ against the preceding element in L.

For example, as shown in Figure 6, when scanning MDMS $M[8, 10]$ against a list containing $M[1, 7]$, its `prev` pointer links to the dummy head. This instantly tells the algorithm that $M[1, 7]$ cannot merge with any subsequent pTMS. When MDMS $M[11, 13]$ is formed, we only need to check whether it is mergeable with $M[8, 10]$, bypassing the check against $M[1, 7]$.

Complexity analysis. Algorithm 1 scans the n objects in the query window to construct $|\mathcal{P}|$ partitions (where $|\mathcal{P}| < n$). This costs $O(n)$ time and $O(n)$ space. Algorithm 2 first spends $O(|\mathcal{P}| \cdot log\ k)$ time to find top-k promising partitions. It then traverses the AVL tree, which costs $O(k \cdot log\ k)$ time. During this process, Algorithm 3 is called to identify maximal intervals within each partition. Assuming partitions are distributed uniformly, each partition P_i contains $\frac{n}{|\mathcal{P}|}$ objects and $|CM_i|$ maximal intervals. Algorithm 3 is linear, so identifying them costs $O(\frac{n}{|\mathcal{P}|})$ time. They participate in comparison, requiring $O(|CM_i| \cdot log\ k)$ time. The total time complexity is $O\big(n + |p| \cdot log\ k + k \cdot log\ k + k \cdot (\frac{n}{|\mathcal{P}|} + |CM_i| \cdot log\ k)\big)$ $= O\big((1 + \frac{k}{|\mathcal{P}|}) \cdot n + (|P| + k + |CM|) \cdot log\ k\big)$.

For space complexity, Algorithm 1 maintains n M values, $|\mathcal{P}|$ partitions including their boundaries, $\mathcal{LM}$ set, and cutting points. Algorithm 2 maintains an AVL tree of size k and the candidate set $\mathcal{CM}$. Algorithm 3 processes a partition requiring space bounded by $O(\frac{n}{|\mathcal{P}|})$; when it completes a partition, the space can be released. Thus, the total space is bounded by $O(n + |\mathcal{P}| + k + |\mathcal{CM}| + \frac{n}{|\mathcal{P}|})$.

5 THE INCREMENTAL MAINTENANCE

Our partitioning strategy is designed not only to support C-KMaxI search within a single window but also to incrementally update the $\mathcal{LM}$ and $\mathcal{CM}$ sets, which are essential for identifying promising partitions and searching for results in the new window. In the following, we first explain how to update the partitions and then detail the maintenance of the $\mathcal{LM}$ and $\mathcal{CM}$ sets corresponding to the updated window.

5.1 Update Partitions

Sliding the window can significantly alter partitions. As shown in Figure 7, when W_1 slides to W_2, partitions evolve: P_2 shrinks, P_4 extends, and P_4' emerges. These changes result from updated cutting points, which shift from $\{o_5, o_{12}, o_{14}, o_{19}\}$ to $\{o_{12}, o_{14}, o_{22}\}$. A naive approach would track these changes by recomputing all affected M values via a full window scan. However, this is highly inefficient,

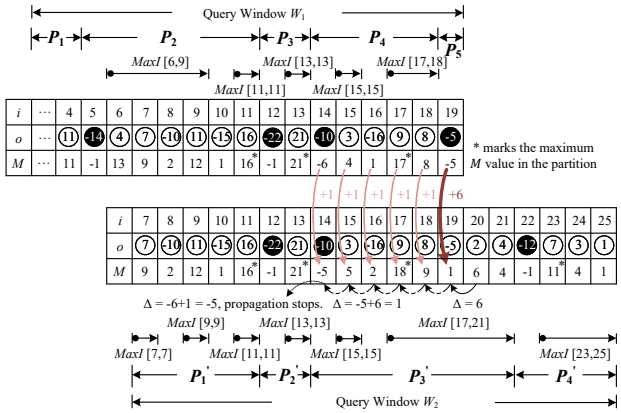

Figure 7: An example of query window sliding from W_1 to W_2.

as many objects' M values remain unchanged and many updated partitions do not contribute to query results. To eliminate these unnecessary computations, we propose a maintenance strategy that updates only a subset of the M values.

Each time the window W advances, the first s objects expire and s new objects arrive. We first calculate the M values for the new objects, in reverse order, from $M[n+s]$ to $M[n+1]$, using Equation (1). If $\Delta = M[n+1] \leq 0$, the update is complete, as this non-positive value blocks further propagation to preceding M values. Otherwise, we propagate the update backward through the existing partitions. This backward propagation process partitions in reverse order, starting from the last partition P_p in W_1. For each partition $P_i = [l_i, r_i]$, if $\Delta_i = M[r_i+1] > 0$, we add this value Δ_i to all M values within P_i. We then proceed to partition P_{i-1}. The propagation terminates at the first partition P_j where $\Delta_j = M[r_j + 1] \leq 0$, as this cutting point blocks the cumulative impact from propagating further. Please take note that all partitions, including degenerate partitions that can be easily recovered from cutting points, participate in this process.

Figure 7 illustrates this process. When W_1 advances to W_2 by $s = 6$ objects, we first calculate the M values for the new objects, i.e., $M[20, 25]$. Since $\Delta = M[20] = 6 > 0$, we propagate the update backward through the partitions: 1) Partition P_5: the M value is increased by Δ. 2) Partition P_4: As $\Delta_4 = M[19] = 1 > 0$, propagation continues and M values are increased by Δ_4. 3) Partition P_3: Since $\Delta_3 = M[14] = -5$, the propagation terminates, leaving M values for objects in partitions P_3, P_2, and P_1 unchanged.

We want to highlight that when propagating the update backward through the existing partitions, Δ value reduces monotonically, as stated in Lemma 7. The proof is skipped, as it can be derived by the fact that $\Delta_i = \Delta_{i+1} + M[r_i+1]$ and $M[r_i+1] = M[l_{i+1}] < 0$ (position l_{i+1} is a cutting point). In addition, a non-cutting point in the original window will not become a cutting point in the updated window, as the M values of objects in a partition P_i change only when the corresponding $\Delta_i > 0$. A non-cutting point has a positive M value, and this value can only increase (or remain unchanged) during backward propagation. Consequently, it cannot become a cutting point, as stated in Lemma 8.

Algorithm 4: Partition Update

Input: Cutting points C, expired objects $\mathcal{E}$, newly arrived objects $\mathcal{A}$, M array, $\mathcal{LM}$ list
Output: Updated cutting points C', and updated $\mathcal{LM}'$ list

$C' \leftarrow C \setminus \mathcal{E}$;
$M_{new}, C_{new} \leftarrow$ `DP-BuildNewCPs`($\mathcal{A}$) ; `// call Algorithm 1`
$C' \leftarrow C' \cup C_{new} \setminus$ `BackwardPropagation`(C', M, M_{new});
$\mathcal{LM}' \leftarrow$ `UpdateLM`($C', C, \mathcal{LM}, M, M_{new}$);
return $C', \mathcal{LM}'$;

Lemma 7. *Let P_i and P_{i+1} be two consecutive partitions whose M values are updated by backward propagation, with Δ_i and Δ_{i+1} representing the corresponding changes. It is confirmed that $\Delta_i < \Delta_{i+1}$.*

Lemma 8. *When the window advances from W to W', let C and C' indicate the sets of cutting points in W and W' respectively, and let $\mathcal{E}$ and $\mathcal{A}$ denote the sets of expired and newly arrived objects. A non-cutting point in W will certainly not become a cutting point in C'. Moreover, C' is a subset of $(C \setminus \mathcal{E}) \cup \mathcal{A}$.*

After the M values are updated, cutting points can be identified to form partitions for the new window. In our example shown in Figure 7, given cutting points o_{12}, o_{14}, and o_{22}, four partitions are formed: $P_1' = [7, 11]$, $P_2' = [12, 13]$, $P_3' = [14, 21]$, and $P_4' = [22, 25]$. We observe that all partition evolutions can be characterized by four fundamental patterns, illustrated in Figure 7: (*i*) Shrinking: a partition loses objects from its head but does not incorporate new ones (e.g., P_1'); (*ii*) Unaltered: a partition remains entirely unaltered (e.g., P_2'); (*iii*) Extended: a partition extends its range by shifting its right boundary to a succeeding position (e.g., P_3'); or (*iv*) Emergent: a new partition forms entirely from newly arrived objects (e.g., P_4'). Note that if a shrinking partition exists, it must be the first partition in the new window. All partition changes conform to one of these four patterns. This is guaranteed because a non-cutting point can never become a cutting point after a window slides, as stated in Lemma 8. However, there is one exception: in an extreme case, the first partition in the new window may both lose initial objects (due to object expiry) and extend its range, although this occurs with very low probability. This would require backward propagation to reach the first cutting point, which is highly unlikely. To simplify our presentation, our algorithm focuses on partition changes that follow one of the four distinct patterns described above. The extreme case can be treated as a Shrinking followed by an Extension.

[zzs says:Revision: replace previous sentence] However, there is one exception: in an extreme case, the first partition in the new window may both lose initial objects (due to object expiry) and extend its range. To simplify our presentation, our algorithm focuses on partition changes that follow one of the four distinct patterns described above. This case can be treated as a Shrinking followed by an Extension. This corner case affects only how we categorize partition evolution for maintenance; it does not affect the correctness of the returned top-k intervals.

Algorithm 4 outlines the procedure. First, expired cutting points are removed. Then, Algorithm 1 is called to process the new objects and identify any new cutting points. The `BackwardPropagation` function is invoked to propagate their cumulative impact backward, returning the set of cutting points whose M values have now become positive. Those replaced cutting points are removed. Finally, we invoke function `UpdateLM` to update LM', the index of top-1 M

value in each partition P' in the new window. Let $P' = [l', r']$ be an updated partition in the new window. We detail its LM' update operation below, dependent on the evolution pattern of P'. i) Shrinking (from partition P_i): If $l' \leq LM_i$, $LM' = LM_i$; otherwise, we scan M values of objects in P' to locate LM'. ii) Unaltered (same as partition P_i in previous window): $LM' = LM_i$. iii) Extended: assume P' covers $P_i, \cdots, P_j$ $(j \geq i)$ in the original window, and possibly including new objects $\mathcal{A}' \subseteq \mathcal{A}$. Due to the nature of backward propagation, the position LM_i within any partition P_i in W continues to have the largest M value among all objects in that partition, even after their M values are updated. Consequently, LM' must lie among the positions $\{LM_i, \cdots, LM_j\}$, and, when $\mathcal{A}' \neq \emptyset$, among the newly arrived objects $\{n+1, \cdots, n+|\mathcal{A}'|\}$, where n indicates the right boundary of the previous window W. We identify LM' by scanning the M values at these positions. iv) Emergent: we scan M values of objects in P' to locate LM'.

Implementation Optimization. In the above description, we assume that the backward propagation stops only after encountering a cutting point say c_i with non-positive M value. This is mainly for ease of explanation. In practice, however, it is not necessary to update the M values for all objects located after c_i. In our implementation, we adopt a lazy update strategy. We update the M values only for new objects and for existing cutting points associated with previous window. The M values of objects within a partition are updated only when necessary (e.g., when determining LM' for a shrinking partition). A binary flag associated with each partition records whether the M values of its objects are updated. When a window slides, the flags of all the partitions are initialized to 0.

Complexity analysis. Removing expired cutting points requires $O(|\mathcal{E}|)$ time. Constructing new cutting points from the newly arrived objects costs $O(|\mathcal{A}|)$ time. Backward impact propagation only updates cutting points' M values rather than all, which requires $O(|C|)$. The total time complexity is $O(|\mathcal{E}| + |\mathcal{A}| + |C|)$. We need auxiliary space to record the cutting points and the M values of new objects, so the total space complexity is $O(|C| + |C_{new}| + |\mathcal{A}|)$.

5.2 Maintain Maximal Intervals Incrementally

Once the partitions and the $\mathcal{LM}$ list are updated, we can answer C-KMaxI query using the partition-based k-maximal interval search (Algorithm 2). Recall that LM values in $\mathcal{LM}$ list identify only the k partitions with the largest top-1 maximal intervals. During the AVL-tree traversal, however, we must still retrieve the other maximal intervals (beyond the top-1) from each promising partition whose top-1 enters the current top-k. A naive strategy would be to compute all maximal intervals in these partitions using Algorithm 3, but this is inefficient because it ignores maximal intervals already obtained in the previous window (maintained in set $\mathcal{CM}$). Depending on how partitions evolve, many of those intervals in $\mathcal{CM}$ remain valid and can be reused. To avoid redundant computation, we incrementally maintain maximal intervals by exploiting partition evolution patterns and reusing prior results. Algorithm 5 summarizes this procedure: it first determines the evolution pattern of the current partition and the availability of reusable maximal intervals, then applies the corresponding maintenance strategy.

Algorithm 5: Incremental Maximal Interval Maintenance

Input: Partition $P' = [l', r']$ in new window with LM' indicating the top-1 M value, original partitions $\mathcal{P} = \{P_1, \cdots, P_p\}$, original $\mathcal{LM}$ list, reusable maximal intervals $\mathcal{CM} = \{CM_1, \cdots, CM_p\}$
Output: Maximal intervals CM' in partition P'

1 **if** $\exists P_j \in \mathcal{P}$, `Unaltered`$(P', P_j) \wedge$ `MaxIReusable`(P_j) **then**
2 $\quad CM' \leftarrow CM_j$;
3 **else if** $\exists P_j \in \mathcal{P}$, `Shrinking`$(P', P_j) \wedge$ `MaxIReusable`(P_j) **then**
4 $\quad CM' \leftarrow CM_j \setminus \{$expired $MaxI[a,b]$ with $b < l'\}$;
5 $\quad$ **if** $\exists\, MaxI[a,b] \in CM'$ *such that* $l' \in (a,b]$ **then**
6 $\quad\quad CM'$.`remove`$(MaxI[a,b])$;
7 $\quad\quad CM'$.`add`(reidentify $MaxI$ within $[l', b]$ via Algorithm 3);
8 **else if** $\exists P_i, \cdots, P_j \in \mathcal{P}$, `Extended`$(P', P_i, \cdots, P_j)$ **then**
9 $\quad$ **if** $\exists a \in [i, j]$, $LM_a = LM' \wedge$ `MaxIReusable`$(P_i, \cdots, P_a)$ **then** `// Case 1`
10 $\quad\quad CM' \leftarrow \cup_{t=i}^{a} CM_t \cup MaxI[LM', r'] \setminus \{MaxI$ in $[LM', r']\}$;
11 $\quad$ **else if** $LM' > n \wedge$ `MaxIReusable`$(P_i, \cdots, P_j)$ **then** `// Case 2`
12 $\quad\quad CM' \leftarrow MaxI[LM', r'] \cup$ reidentify $MaxI$ within $[l', LM' - 1]$ via Algorithm 3;
13 **else** `// emergent partition or partitions without reusable MaxI`
14 $\quad CM' \leftarrow$ Identify Maximal Interval via Algorithm 3;
15 **return** CM';

Specifically, for new partitions or partitions without reusable maximal intervals (Lines 13-14), maximal intervals must be computed from scratch using Algorithm 3. When reusable maximal intervals exist (determined by function `MaxIReusable`), unaltered partitions can directly reuse all previously computed maximal intervals (Lines 1-2). For shrinking and extended partitions, the corresponding maintenance strategies are detailed below.

Shrinking Partitions. Let partition $P' = [l', r]$ be a shrinking partition derived from partition $P = [l, r]$, where $l' > l$. The maximal intervals containing only expired objects are removed. If position $l' - 1$ falls within an existing maximal interval $MaxI[a, b]$, we need to perform the following updates. First, $MaxI[a, b]$ shrinks to $I[l', b]$. It is guaranteed that pTMS $M[l', b+1]$ corresponding to interval $I[l', b]$ will not merge with any other pTMS in this partition. This is because in the maximal interval $MaxI[a, b]$, $M[a]$ has the strict maximal M value, and therefore $M[a] > M[l']$. The fact that $MaxI[a, b]$ is a maximal interval in $P = [l, r]$ and $M[a] > M[l']$ guarantees that $M[l', b+1]$ cannot merge with any other pTMS in this partition. Next, we invoke Algorithm 3 to identify maximal intervals, if any, in the range of $[l', b]$.

In summary, in a shrinking partition, maximal intervals containing only expired objects are removed, the maximal interval containing both expired and valid objects, if it exists, requires further evaluation, and all other maximal intervals remain unchanged. For example, in Figure 7, partition P_2 in W_1 shrinks to P_1' in W_2. Objects o_5 and o_6 in P_2 expire, affecting only maximal interval $MaxI[6, 9]$, which shrinks to interval $I[7, 9]$. We then locate two maximal intervals $I[7, 7]$ and $I[9, 9]$ in this reduced range via Algorithm 3. The other maximal interval, $MaxI[11, 11]$, remains unchanged.

Extended Partitions. Let partition $P' = [l_i, r']$ be an extended partition originating from $P_i = [l_i, r_i]$ $(r' > r_i)$, covering the sequence of partitions $P_i, \cdots, P_j$ for some $j \geq i$, and possibly including new objects $\mathcal{A}' \subseteq \mathcal{A}$. Let LM' denote the new position in P' with the largest M value. The maintenance operations depend on whether $LM' > n$, where n denotes the ending position of the previous window W, as described below.

Case 1): $LM' \leq n$. This means that the position with the top-1 M value lies within an existing partition, say P_a (i.e., $LM' = LM_a$). Accordingly, $MaxI[LM', r']$ is the top-1 maximal interval in the extended partition P', and all maximal intervals within this range become invalid and are removed. However, maximal intervals that end before position LM' remain unchanged. In other words, if CMs corresponding to $P_i, \cdots, P_a$ are available, we can immediately identify CM' by re-using all maximal intervals ending before LM_a. Otherwise, we only need to compute CMs of those partitions among $P_i, \cdots, P_a$ whose CMs are unavailable. We did not explicitly highlight this in our pseudo-code due to space constraints and for code simplicity. Consider the extended partition $P'_3 = [14, 21]$ in Figure 7 as an example. P'_3 (with $LM' = 17$) originates from P_4, covers partitions P_4 and P_5, and includes new objects $\mathcal{A}' = \{o_{20}, o_{21}\}$. Accordingly, $MaxI[17, 21]$ becomes the new top-1 maximal interval, and the previous maximal interval $MaxI[17, 18]$ is removed. All maximal intervals ending before position 17 (i.e., $MaxI[15, 15]$) remain valid.

Case 2): $LM' > n$. This means the position with the top-1 M value is located among the new objects. $I[LM', r']$ becomes the top-1 maximal interval in the extended partition P', and we need to relocate the maximal intervals in the range of $[l_i, LM' - 1]$ via Algorithm 3.

Complexity analysis. Let $|\mathcal{P}|$ partitions with a uniform distribution exist in the window with n objects. In Algorithm 5, only when the worst case (case 2 in extended partitions) occurs, maximal intervals in the partition need to be re-identified. In this case, the algorithm degenerates to the scenario where Algorithm 3 processes one partition, which requires $O(\frac{n}{|\mathcal{P}|})$ time and $O(\frac{n}{|\mathcal{P}|})$ space.

6 EXPERIMENTS

6.1 Experiment Setting

Datasets. We conduct our experiments on five datasets: three real datasets, Stock, Power, and Genome, and two synthetic datasets, Uniform and Normal. 1) Stock. Bitcoin price records from Alpha Vantage [1], spanning from Nov. 4, 2013, to Mar. 31, 2025. After preprocessing to retain 5-minute open/close prices, the dataset contains 9M records. [zzs says:Revision: replace previous sentence]After preprocessing as 5-minute price delta stream (observation price = open price − close price), the dataset contains 9M records, and the top-k maximal intervals mean periods with the greatest price increase. 2) Power. UK's half-hourly power data (Nov. 2008 - Dec. 2022) from the Elexon Portal and National Grid, compiled by Kaggle [2]. We calculate the total power generation and consumption, and their difference forms a power surplus sequence with 0.24M records. [zzs says:Revision: replace previous sentence]We compute the differences of the power generation and consumption to form a power surplus observation stream with 0.24M records, and the top-k maximal intervals mean periods with the greatest power cumulative surplus. 3) Genome. The Haemophilus influenzae genome sequence from NCBI [3]. Following prior work [27], each CpG dinucleotide receives a score of 20, and all other −1, producing a score sequence of 1.8M records. 4) Uniform, and 5) Normal. Synthetic datasets of 10M real numbers each, generated according to uniform and normal distributions.

Table 2: Parameter Setting

Parameters	Values
n	0.1%, 0.5%, <u>1%</u>, 5%, 10% ($\times\|D\|$)
s	0.01%, <u>0.1%</u>, 0.5%, 1%, 10% ($\times n$)
k	5, 10, <u>50</u>, 100, 1000

Parameter Setting. We evaluate algorithm performance by varying three key parameters: n, the number of objects in the window; s, the number of objects entering or expiring from the window per slide; and k, the number of maximal intervals queried. For each dataset, we run five queries using the n, s, and k values in Table 2. Parameters not under variation are fixed at their underlined default values. This ensures a controlled and fair assessment of each parameter's impact. [zzs says:Revision: add these sentences.] In practice, these parameters are determined by application requirements (e.g., how much history must be monitored, update frequency, and how many disjoint "most significant" periods are needed), and there is no single universally optimal configuration. Therefore, following common practice in streaming and sliding-window evaluation [9, 24, 28, 31–34, 36–38], we choose parameter ranges that (i) cover multiple orders of magnitude, (ii) are normalized across datasets for comparability, and (iii) stress scalability under different workloads. Using percentages avoids overfitting parameter choices to a specific dataset scale and makes results comparable across datasets with different cardinalities. The default (n, s, k) represents a representative mid-range streaming workload with substantial window overlap and a moderate number of requested disjoint intervals.

Performance Metrics. Following prior evaluation methodologies [9, 24, 28, 31–34, 36–38], we assess algorithm performance using four metrics: 1) *Overall running time*: total time required for processing both queries and updates. 2) *Data throughput*: average number of updates processed per second, reflecting efficiency in dynamic data streams. 3) *Candidate set size*: average number of maximal intervals in the candidate set, demonstrating the effectiveness of the selection criteria. 4) *Memory usage*: comparison of memory consumption across all algorithms and datasets.

Competitors. We implemented our proposed partition-based top-k maximal intervals search algorithm. We also implemented two existing algorithms, Ruzzo-Tompa (RT) [27] and Tournament [7]. To adapt these algorithms to the streaming context, we further developed their streaming versions that support incremental maintenance: Streaming RT and Streaming Tournament, [zzs says:which are detailed in Appendixes A and B]. All algorithms are implemented in C++ and evaluated on a PC running Windows 11 with an Intel Core i5-13500H CPU and 32GB main memory.

6.2 Algorithm Comparisons

Running time comparison. We begin by assessing the running time of all algorithms under varying data scales, ranging from 20% to 100% of each dataset. For example, using 20% of STOCK means that 20% of the original records are selected to form a new dataset containing approximately $9 \times 0.2 = 1.8$M records. The running time required to process C-KMaxI while simultaneously updating partitions is reported in Figure 8. The results clearly demonstrate that the partition-based approach consistently outperforms all competing algorithms across all datasets, achieving substantial performance

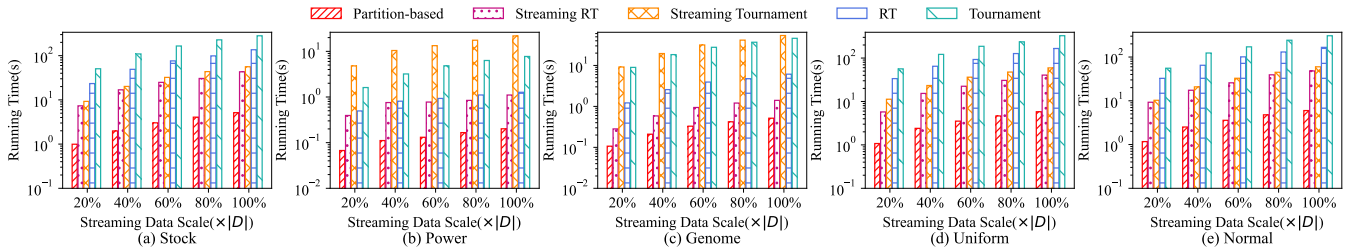

Figure 8: Running time comparison under different data scales.

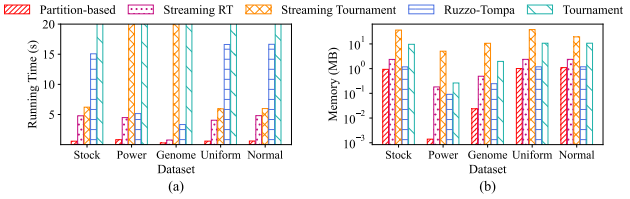

Figure 9: (a) Running time comparison processing one million objects under different datasets. (b) Memory usage comparison under different datasets.

gains. For example, on the Stock dataset, it achieves speedups of 8× and 10× over Streaming RT and Streaming Tournament, respectively. Additionally, it outperforms the original RT and Tournament by more than an order of magnitude. Similar advantages are observed across the remaining datasets, where the partition-based algorithm exhibits consistent and often significant superiority.

The superior performance of our method can be primarily attributed to the proposed partitioning strategy. Compared with competing algorithms, its advantages are evident in three key aspects. First, unlike Streaming RT, our method proactively identifies partitions likely to contain query results, allowing maximal interval identification and comparison only within these partitions, thereby substantially reducing the solution space. Second, unlike Streaming Tournament, our method does not rely on index structures and therefore avoids the computational overhead of index restructuring. Third, our approach incorporates advanced incremental maintenance techniques. The partition-based incremental maintenance accurately identifies the affected range and maximizes the reuse of previously computed maximal intervals. In contrast, although both Streaming RT and Streaming Tournament support incremental maintenance, Streaming RT incurs a larger affected range due to the absence of partitions, while Streaming Tournament frequently requires expensive structural adjustments. Furthermore, the original RT and Tournament lack dedicated incremental maintenance mechanisms for streaming settings and must re-identify all maximal intervals within each query window, resulting in significantly lower efficiency.

We also observe that the performance gap varies considerably across datasets. For example, on the Stock dataset, the partition-based approach outperforms Streaming Tournament by approximately an order of magnitude on average, whereas its advantage increases to nearly two orders of magnitude on the Power and Genome datasets.

To further examine the sensitivity of the algorithms to data distribution, we analyze the running time required by each algorithm to process one million objects across multiple datasets. The results are presented in Figure 9(a). We find that, for each algorithm, the time needed to process one million updates varies across datasets. This variation arises because maximal intervals, the targets of interest, are inherently sensitive to the underlying data distribution. As subsequence of real numbers, both their length and quantity are influenced by distributional characteristics.

However, it is evident that our approach exhibits the least sensitivity to data distribution. This robustness stems from the partitioning strategy: although the length and number of maximal intervals may fluctuate significantly across datasets, our method processes only a small subset of the data, specifically at most k partitions most likely to contain query results. This effectively mitigates dataset-specific variations and reduces the impact of distributional differences. Consequently, we conclude that our approach demonstrates notably greater stability compared to all competing algorithms.

Data throughput comparison. Figure 10 presents the data throughput of all algorithms under varying parameter settings across different datasets, showing that our approach consistently outperforms all competitors by a significant margin. As shown in Figures 10(a)-(e), as n increases, the throughput of both our approach and Streaming Tournament improves, while the throughput of the other algorithms remains relatively stable. Note, due to the small size of the Power dataset, its n value starts at 0.5% rather than 0.1%. A larger n indicates that the query window contains more objects and thus more maximal intervals. However, our approach continues to process only the partitions likely to contain query results, while Streaming Tournament directly searches for the top-k results based on the index. Consequently, the number of updates processed per unit time increases for both methods.

Conversely, Figures 10(k)-(o) show that as k increases, the throughput of the partition-based algorithm experiences a slight decrease, while that of Streaming Tournament decreases sharply. Meanwhile, the throughput of the other algorithms remains relatively stable. This behavior arises because the partition-based algorithm must process more partitions as k grows, whereas Streaming Tournament performs more queries. Both RT and Streaming RT retain all maximal intervals, and Tournament incurs substantial overhead due to frequent index restructuring. As a result, their performance is less sensitive to k. Importantly, even when k is large, the partition-based algorithm still maintains a clear performance advantage.

Figures 10(f)-(j) illustrate that all algorithms exhibit increased throughput as s becomes larger. When s is small, there is considerable overlap between windows, causing many objects to be

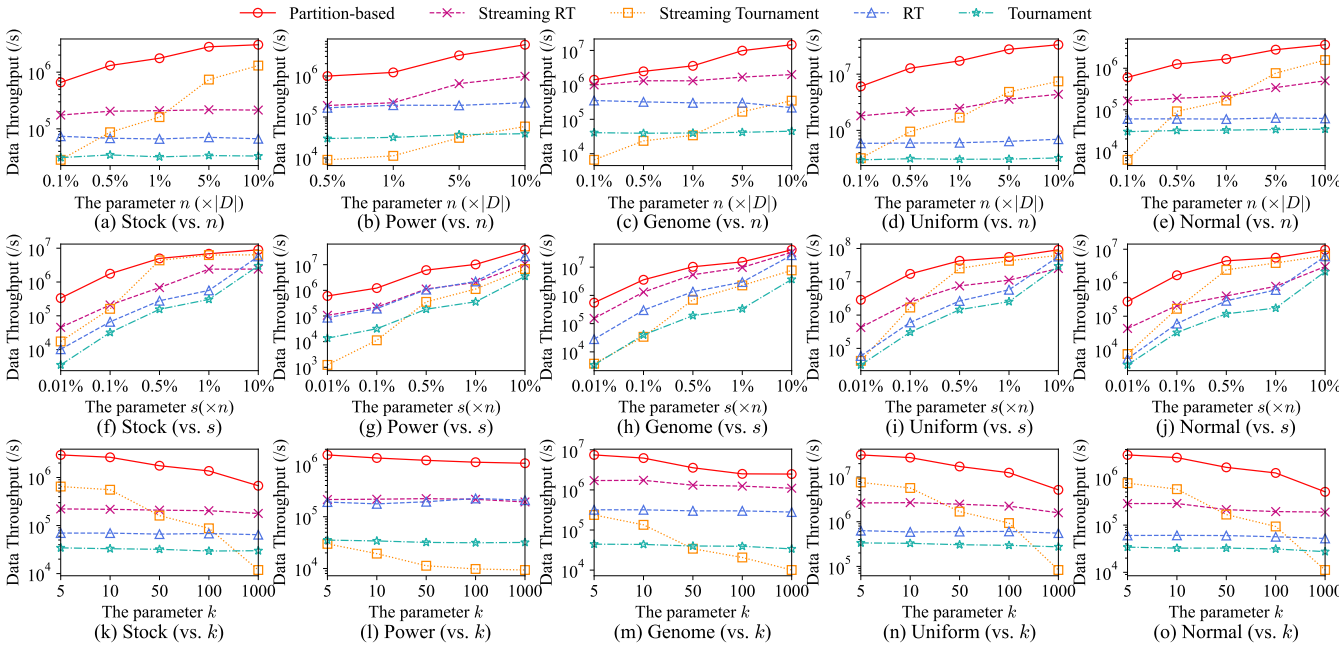

Figure 10: Data throughput comparison under different datasets.

Table 3: The size of the candidate set (UNIT:*count*)

Solutions	Stock	Power	Genome	Uniform	Normal
Partition-based	930	16	194	1270	1252
Streaming RT	1927	17	207	2961	3032
RT	1927	17	207	2961	3032

recomputed repeatedly. As s increases, the overhead from redundant computations is reduced, leading to an overall improvement in throughput.

Candidate set size. Table 3 reports the average number of maximal intervals maintained in the candidate set. The two Tournament-Tree-based-approaches do not rely on a candidate set and are therefore excluded from this statistic. Both Streaming RT and RT maintain all maximal intervals, so their values in the table reflect the average number of maximal intervals per window. In contrast, our approach significantly reduces the number of intervals in the candidate set. This improvement arises because we incrementally maintain maximal intervals only within the partitions likely to contain query results. The Power dataset is the only exception: our method reduces candidate set only from 17 to 16, because this dataset inherently produces very few maximal intervals, leaving limited room for enhancement.

Table 4 presents the average per-window statistics for candidate identification (id.) and usage (us.). "id." denotes the number of maximal intervals identified, while "us." counts how many times these intervals are used, including comparisons for top-k selection and candidate-set maintenance. The results show that our approach achieves substantial improvements in both metrics. This advantage stems from our partitioning strategy: during maximal interval identification, top-k comparison, and incremental maintenance, the strategy avoids exhaustively processing all maximal intervals and instead focuses only on partitions likely to contain query results.

Memory usage comparison. Figure 9(b) illustrates the maximum memory usage of all the algorithms during their execution across

Table 4: Average identification and usage statistics of candidates (UNIT:*count* $\times$ *times*)

Dataset		Partition-based	Streaming RT	RT
Stock	id.	49.29	1244.15	39401.89
	us.	370.82	1927.74	1927.74
Power	id.	0.51	92.15	431.62
	us.	0.97	17.13	17.13
Genome	id.	1.67	6.90	724.41
	us.	6.30	207.73	207.73
Uniform	id.	50.74	1228.59	49977.34
	us.	569.54	2961.59	2961.59
Normal	id.	51.83	1137.16	49972.99
	us.	620.57	3032.76	3032.76

various datasets. Among these, the partition-based approach consumes the least memory, as the partitioning strategy minimizes the number of partitions processed. Both Streaming RT and RT require comparing all maximal intervals to select the top-k, necessitating the use of more memory to identify and maintain all maximal intervals. While Streaming Tournament and Tournament do not maintain a candidate set, they still incur higher memory costs compared to the other algorithms due to the construction of the Tournament tree. Furthermore, the memory usage of the partition-based approach is notably lower than that of the other algorithms, particularly on the Power dataset. A detailed examination of this dataset reveals that most maximal intervals exist in the form of the maximal interval with the largest sum within its respective partition. These intervals can be directly obtained during partition construction, eliminating the need to invoke the maximal interval identification algorithm (Algorithm 3), which results in a substantial reduction in memory usage.

6.3 The evaluation experiment for the impact of the number of partitions

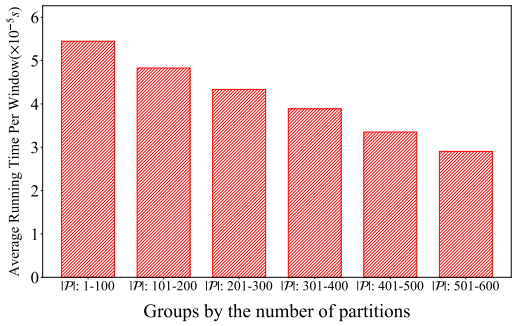

Figure 11: Average running time per sliding window under different numbers of partitions.

Our partitioning scheme is data-driven: for a fixed window content, the cutting points and thus the number of partitions $|\mathcal{P}|$ are uniquely determined. To evaluate the impact of partition count in practice, we perform a window stratification study under the default parameters. we process the entire stream and record $|\mathcal{P}|$ for every sliding window. We then equally group windows into six groups by $|\mathcal{P}|$: windows containing 0–100 partitions belong to the first group, the second group includes windows containing 101–200 partitions, $\cdots$ We report the average running time per window in each group on the Stock stream, shown in Figure 11, but we skip the results on other datasets for space saving.

The results exhibit a clear monotonic trend: windows with larger $|\mathcal{P}|$ have lower average running time. This is consistent with the core benefit of our partition-based framework: more partitions provide finer-grained localization and pruning, so the top-k search focuses on fewer irrelevant regions, reducing overall work. Meanwhile, the overhead of maintaining additional partitions is relatively small compared with the savings from the reduced search space, indicating that our method is robust across a wide range of partition numbers.

6.4 The correctness validation experiment

To further validate the correctness of our implementation, we conduct a correctness validation experiment using the *Exact Match Rate* as the evaluation metric. In this experiment, we compare the top-k maximal sum intervals returned by our framework with the ground truth generated by the original Ruzzo-Tompa (RT) algorithm (Algorithm 6), which recomputes all maximal intervals from scratch for each window. For every sliding window under the default parameter configuration and across all five datasets (Stock, Power, Genome, Uniform, and Normal), we compute whether the returned top-k intervals exactly match those produced by RT. A window is counted as a match only if both the selected intervals and their ranking order are identical to the ground truth.

Table 5 reports the Exact Match Rate over all sliding windows. As shown, the Exact Match Rate is 100% on every dataset. This result confirms that our partition-based framework produces outputs that are fully consistent with the exhaustive RT algorithm.

Table 5: Correctness validation using Exact Match Rate (RT as ground truth). All sliding windows are evaluated.

Dataset	Stock	Power	Genome	Uniform	Normal
Exact Match Rate (%)	100	100	100	100	100

7 CONCLUSION

This paper introduces an innovative partitioning strategy designed to support C-KMaxI over streaming data. The proposed strategy demonstrates significant efficiency in candidate identification, candidate set maintenance, and query result retrieval. Its adaptability enables effective support for C-KMaxI using a compact candidate set. Our comprehensive experiments across diverse datasets validate the superior performance and relative stability of the proposed algorithms. In the near future, we would like to study parallel algorithms for C-KMaxI over streaming data, leveraging the inherent property that partitions can be processed independently.

REFERENCES

[1] 2025. *Alpha Vantage.* https://www.alphavantage.co/ Accessed: October 15, 2025.

[2] 2025. *Electrical Grid Half Hourly (UK).* https://www.kaggle.com/datasets/thedevastator/gb-electrical-grid-half-hourly-data-2008-present Accessed: October 15, 2025.

[3] 2025. *Haemophilus influenzae whole genome.* https://www.ncbi.nlm.nih.gov/Traces/wgs/JAMLFG01 Accessed: October 15, 2025.

[4] 2025. *Maximum subarray problem.* https://en.wikipedia.org/wiki/Maximum_subarray_problem Accessed: October 6, 2025.

[5] 2026. *Ruzzo-Tompa algorithm.* https://en.wikipedia.org/wiki/Ruzzo%E2%80%93Tompa_algorithm Accessed: Feb. 28, 2026.

[6] Miklós Ajtai, T. S. Jayram, Ravi Kumar, and D. Sivakumar. 2002. Approximate counting of inversions in a data stream. In *Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada*, John H. Reif (Ed.). ACM, 370–379. https://doi.org/10.1145/509907.509964

[7] Sung Eun Bae and Tadao Takaoka. 2007. Algorithms for k-Disjoint Maximum Subarrays. *Int. J. Found. Comput. Sci.* 18, 2 (2007), 319–339. https://doi.org/10.1142/S012905410700470X

[8] Fredrik Bengtsson and Jingsen Chen. 2006. Computing Maximum-Scoring Segments in Almost Linear Time. In *Computing and Combinatorics, 12th Annual International Conference, COCOON 2006, Taipei, Taiwan, August 15-18, 2006, Proceedings (Lecture Notes in Computer Science)*, Danny Z. Chen and D. T. Lee (Eds.), Vol. 4112. Springer, 255–264. https://doi.org/10.1007/11809678_28

[9] Christian Böhm, Beng Chin Ooi, Claudia Plant, and Ying Yan. 2007. Efficiently Processing Continuous k-NN Queries on Data Streams. In *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007*, Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis (Eds.). IEEE Computer Society, 156–165. https://doi.org/10.1109/ICDE.2007.367861

[10] Ben Chen, Zhijin Lv, Xiaohui Yu, and Yang Liu. 2017. Sliding Window Top-K Monitoring over Distributed Data Streams. *Data Sci. Eng.* 2, 4 (2017), 289–300. https://doi.org/10.1007/S41019-017-0053-1

[11] Miklós Csürös. 2004. Maximum-Scoring Segment Sets. *IEEE ACM Trans. Comput. Biol. Bioinform.* 1, 4 (2004), 139–150. https://doi.org/10.1109/TCBB.2004.43

[12] Eros D. Escobar, Daniel Betancur, and Idi A. Isaac. 2024. Optimal Power and Battery Storage Dispatch Architecture for Microgrids: Implementation in a Campus Microgrid. *Smart Grids and Sustainable Energy* 9, 2 (2024), 27. https://doi.org/10.1007/s40866-024-00210-8

[13] Pawel Gawrychowski and Patrick K. Nicholson. 2015. Encodings of Range Maximum-Sum Segment Queries and Applications. In *Combinatorial Pattern Matching - 26th Annual Symposium, CPM 2015, Ischia Island, Italy, June 29 - July 1, 2015, Proceedings (Lecture Notes in Computer Science)*, Ferdinando Cicalese, Ely Porat, and Ugo Vaccaro (Eds.), Vol. 9133. Springer, 196–206. https://doi.org/10.1007/978-3-319-19929-0_17

[14] Bugra Gedik, Kun-Lung Wu, Philip S. Yu, and Ling Liu. 2007. A Load Shedding Framework and Optimizations for M-way Windowed Stream Joins. In *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007*, Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis (Eds.). IEEE Computer Society, 536–545. https://doi.org/10.1109/ICDE.2007.367899

[15] Xiaohui Gu, Philip S. Yu, and Haixun Wang. 2007. Adaptive Load Diffusion for Multiway Windowed Stream Joins. In *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007*, Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis (Eds.). IEEE Computer Society, 146–155. https://doi.org/10.1109/ICDE.2007.367860

[16] Parisa Haghani, Sebastian Michel, and Karl Aberer. 2009. Evaluating top-k queries over incomplete data streams. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management, CIKM 2009, Hong Kong, China, November 2-6, 2009*, David Wai-Lok Cheung, Il-Yeol Song, Wesley W. Chu, Xiaohua Hu, and Jimmy Lin (Eds.). ACM, 877–886. https://doi.org/10.1145/1645953.1646064

[17] Cheqing Jin, Ke Yi, Lei Chen, Jeffrey Xu Yu, and Xuemin Lin. 2010. Sliding-window top-k queries on uncertain streams. *VLDB J.* 19, 3 (2010), 411–435. https://doi.org/10.1007/S00778-009-0171-0

[18] Yuan Jin, Yunliang Lai, Azadeh Noori Hoshyar, Nisreen Innab, Meshal Shutaywi, Wejdan Deebani, and A Swathi. 2025. An ideally designed deep trust network model for heart disease prediction based on seagull optimization and Ruzzo Tompa algorithm. *Scientific Reports* 15, 1 (2025), 6035.

[19] S Karlin and S F Altschul. 1993. Applications and statistics for multiple high-scoring segments in molecular sequences. *Proceedings of the National Academy of Sciences* 90, 12 (1993), 5873–5877. https://doi.org/10.1073/pnas.90.12.5873 arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.90.12.5873

[20] Samuel Karlin and Volker Brendel. 1992. Chance and Statistical Significance in Protein and DNA Sequence Analysis. *Science* 257, 5066 (1992), 39–49. https://doi.org/10.1126/science.1621093 arXiv:https://www.science.org/doi/pdf/10.1126/science.1621093

[21] Tianyi Li, Yu Gu, Xiangmin Zhou, Qian Ma, and Ge Yu. 2017. An Effective and Efficient Truth Discovery Framework over Data Streams. In *Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017, Venice, Italy, March 21-24, 2017*, Volker Markl, Salvatore Orlando, Bernhard Mitschang, Periklis Andritsos, Kai-Uwe Sattler, and Sebastian Breß (Eds.). OpenProceedings.org, 180–191. https://doi.org/10.5441/002/EDBT.2017.17

[22] Shangsong Liang, Zhaochun Ren, Wouter Weerkamp, Edgar Meij, and Maarten de Rijke. 2014. Time-Aware Rank Aggregation for Microblog Search. In *Proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management* (Shanghai, China) *(CIKM '14)*. Association for Computing Machinery, New York, NY, USA, 989–998. https://doi.org/10.1145/2661829.2661905

[23] Periklis Mantenoglou, Alexander Artikis, and Georgios Paliouras. 2023. Online event recognition over noisy data streams. *Int. J. Approx. Reason.* 161 (2023), 108993. https://doi.org/10.1016/J.IJAR.2023.108993

[24] Kyriakos Mouratidis, Spiridon Bakiras, and Dimitris Papadias. 2006. Continuous monitoring of top-k queries over sliding windows. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Chicago, Illinois, USA, June 27-29, 2006*, Surajit Chaudhuri, Vagelis Hristidis, and Neoklis Polyzotis (Eds.). ACM, 635–646. https://doi.org/10.1145/1142473.1142544

[25] Jeff Pasternack and Dan Roth. 2009. Extracting article text from the web with maximum subsequence segmentation. In *Proceedings of the 18th International Conference on World Wide Web* (Madrid, Spain) *(WWW '09)*. Association for Computing Machinery, New York, NY, USA, 971–980.

[26] Víctor Rampérez, Shima Zahmatkesh, and Emanuele Della Valle. 2021. Scaling the monitoring of approximate top-k queries in streaming windows. In *2021 IEEE International Conference on Big Data (Big Data), Orlando, FL, USA, December 15-18, 2021*, Yixin Chen, Heiko Ludwig, Yicheng Tu, Usama M. Fayyad, Xingquan Zhu, Xiaohua Hu, Suren Byna, Xiong Liu, Jianping Zhang, Shirui Pan, Vagelis Papalexakis, Jianwu Wang, Alfredo Cuzzocrea, and Carlos Ordonez (Eds.). IEEE, 181–189. https://doi.org/10.1109/BIGDATA52589.2021.9672041

[27] Walter L. Ruzzo and Martin Tompa. 1999. A Linear Time Algorithm for Finding All Maximal Scoring Subsequences. In *Proceedings of the Seventh International Conference on Intelligent Systems for Molecular Biology, August 6-10, 1999, Heidelberg, Germany*, Thomas Lengauer, Reinhard Schneider, Peer Bork, Douglas L. Brutlag, Janice I. Glasgow, Hans-Werner Mewes, and Ralf Zimmer (Eds.). AAAI, 234–241. http://www.aaai.org/Library/ISMB/1999/ismb99-027.php

[28] Avani Shastri, Di Yang, Elke A. Rundensteiner, and Matthew O. Ward. 2011. MTopS: scalable processing of continuous top-k multi-query workloads. In *Proceedings of the 20th ACM Conference on Information and Knowledge Management, CIKM 2011, Glasgow, United Kingdom, October 24-28, 2011*, Craig Macdonald, Iadh Ounis, and Ian Ruthven (Eds.). ACM, 1107–1116. https://doi.org/10.1145/2063576.2063736

[29] John L Spouge, Leonardo Mariño-Ramírez, and Sergey L Sheetlin. 2012. The ruzzo-tompa algorithm can find the maximal paths in weighted, directed graphs on a one-dimensional lattice. In *2012 IEEE 2nd International Conference on Computational Advances in Bio and medical Sciences (ICCABS)*. IEEE, 1–6.

[30] Julien Subercaze, Christophe Gravier, Syed Gillani, Abderrahmen Kammoun, and Frédérique Laforest. 2017. Upsortable: Programming TopK Queries Over Data Streams. *Proc. VLDB Endow.* 10, 12 (2017), 1873–1876. https://doi.org/10.14778/3137765.3137797

[31] Yufei Tao and Dimitris Papadias. 2006. Maintaining Sliding Window Skylines on Data Streams. *IEEE Trans. Knowl. Data Eng.* 18, 2 (2006), 377–391. https://doi.org/10.1109/TKDE.2006.48

[32] Luan V. Tran, Liyue Fan, and Cyrus Shahabi. 2016. Distance-based Outlier Detection in Data Streams. *Proc. VLDB Endow.* 9, 12 (2016), 1089–1100. https://doi.org/10.14778/2994509.2994526

[33] Luan V. Tran, Minyoung Mun, and Cyrus Shahabi. 2020. Real-Time Distance-Based Outlier Detection in Data Streams. *Proc. VLDB Endow.* 14, 2 (2020), 141–153. https://doi.org/10.14778/3425879.3425885

[34] Di Yang, Avani Shastri, Elke A. Rundensteiner, and Matthew O. Ward. 2011. An optimal strategy for monitoring top-k queries in streaming windows. In *EDBT 2011, 14th International Conference on Extending Database Technology, Uppsala, Sweden, March 21-24, 2011, Proceedings*, Anastasia Ailamaki, Sihem Amer-Yahia, Jignesh M. Patel, Tore Risch, Pierre Senellart, and Julia Stoyanovich (Eds.). ACM, 57–68. https://doi.org/10.1145/1951365.1951375

[35] Meng Yang, Rui Xie, Yongjun Zhang, and Yue Chen. 2025. Robust Microgrid Dispatch With Real-Time Energy Sharing and Endogenous Uncertainty. *IEEE Transactions on Smart Grid* 16, 4 (2025), 3085–3098. https://doi.org/10.1109/TSG.2025.3569095

[36] Susik Yoon, Jae-Gil Lee, and Byung Suk Lee. 2019. NETS: Extremely Fast Outlier Detection from a Data Stream via Set-Based Processing. *Proc. VLDB Endow.* 12, 11 (2019), 1303–1315. https://doi.org/10.14778/3342263.3342269

[37] Rui Zhu, Bin Wang, Xiaochun Yang, and Baihua Zheng. 2023. Closest Pairs Search Over Data Stream. *Proc. ACM Manag. Data* 1, 3 (2023), 205:1–205:26. https://doi.org/10.1145/3617326

[38] Rui Zhu, Bin Wang, Xiaochun Yang, Baihua Zheng, and Guoren Wang. 2017. SAP: Improving Continuous Top-K Queries Over Streaming Data. *IEEE Trans. Knowl. Data Eng.* 29, 6 (2017), 1310–1328. https://doi.org/10.1109/TKDE.2017.2662236

A RUZZO-TOMPA ALGORITHM AND ITS STREAMING VERSION

[zzs says:All contents in the appendix are our added revision.] In this section, we introduce our competing algorithm, the Ruzzo-Tompa algorithm, and its Streaming version.

A.1 Ruzzo-Tompa algorithm (static version)

We first introduce the Ruzzo-Tompa algorithm (static version). This algorithm is proposed in the work [27]. Wikipedia [5] presents its implementation using Python. However, the implementation in Wikipedia lacks the optimization of the skip pointer mechanism, which results in $O(n^2)$ time complexity in the worst case. In our experiments, we implement this algorithm with this optimization.

Algorithm 6: Ruzzo–Tompa Algorithm with Skip Pointers

```
Input: Objects o[0, ···, n − 1]
Output: List of maximal subsequences
k ← 0, total ← 0 ;
Initialize arrays I, L, R, Lidx of size n ;
Initialize array skip of size n to NULL // Skip pointer initialization ;
for i ← 0 to n − 1 do
    total ← total + o[i] ;
    if o[i] > 0 then
        I[k] ← (i, i + 1) // Maintain maximal intervals;
        Lidx[k] ← i, L[k] ← total − o[i], R[k] ← total ;
        while True do
            maxj ← None ;
            for j ← k − 1 down to 0 do
                if L[j] < L[k] then
                    maxj ← j ;
                    break ;
            if maxj ≠ None and R[maxj] < R[k] then
                I[maxj] ← (Lidx[maxj], i + 1) ;
                R[maxj] ← total ;
                k ← maxj ;
            else
                skip[k] ← k − 1 // Added skip pointer ;
                k ← k + 1 ;
                break ;
    if skip[k] ≠ NULL then
        k ← skip[k] // Skip to the previous valid subsequence ;
return List of maximal intervals: I[m] for m ∈ 0, ..., k − 1, where each
    I[m] is an interval (L[m], R[m])
```

The Ruzzo-Tompa algorithm, with the optimization of the skip pointer mechanism, finds all maximal intervals in $O(n)$ time and $O(n)$ space, where n is the number of the input objects, outlined in Algorithm 6. The algorithm works by progressively building the solution, maintaining an ordered list of disjoint maximal intervals. Each subsequence I_j is associated with two values: the cumulative sum of all objects up to but not including the leftmost object of I_j, denoted as L_j, and the cumulative sum of all objects up to and including the rightmost object of I_j, denoted as R_j. $Lidx[j]$ records the starting index (i.e., the left boundary) of the current candidate subsequence I_j.

The algorithm proceeds as follows: Objects are read from left to right, maintaining the cumulative sum of the objects encountered so far. If a non-positive object is encountered, no special processing is performed, and the algorithm proceeds to the next object. If a positive object is encountered, a new subsequence I_k is created, including only this object. This subsequence is then incorporated into the list of disjoint subsequences by examining all previous subsequences in the list, from right to left, to find the subsequence I_j that satisfies $L_j < L_k$. If no such subsequence is found, the new subsequence I_k is appended to the end of the list. If a subsequence I_j is found such that $R_j \geq R_k$, then the new subsequence I_k is appended to the end of the list. If a subsequence I_j is found such that $R_j < R_k$, then the subsequence I_k is extended to the left to include all of I_j. The subsequences from I_j to I_k are removed from the list, and the newly extended subsequence is reconsidered and re-inserted into the list. The algorithm continues this process for each object in the input list.

To improve efficiency, a skip pointer mechanism is introduced to optimize the process of finding the appropriate subsequence for a new object. This mechanism reduces redundant comparisons by skipping over subsequences that have already been processed. Specifically:

- The skip pointer array is initialized to `NULL` for all subsequences at the beginning.
- When adding a new subsequence I_k, the algorithm first searches for a valid subsequence I_j to merge with, starting from the most recent subsequence.
- If a valid subsequence I_j is found, but $R_j < R_k$, the skip pointer for the current subsequence I_k is set to point to the previous valid subsequence, allowing the algorithm to skip over unnecessary comparisons in the future.
- The algorithm updates $Lidx[k]$ when a new subsequence is created or extended, ensuring that the left boundary of each subsequence is correctly tracked.
- When processing a new subsequence, if the skip pointer for the current subsequence is not `NULL`, the algorithm jumps to the previously valid subsequence, thus avoiding reprocessing subsequences that have already been considered.

The algorithm terminates once all objects have been processed. The list of subsequences remaining in the list at the end corresponds to the maximal intervals in the input sequence.

A.2 Streaming Ruzzo-Tompa algorithm

We develop an incremental maintenance algorithm for the Ruzzo-Tompa algorithm, detailed in Algorithm 7, enabling it adapt to the streaming context and avoid re-identifying all maximal intervals from scratch as the query window slides.

The key idea of the incremental maintenance is to maintain a candidate set of maximal intervals, $\mathcal{CM}$, which contains the maximal intervals from the previous window. It first handles changes resulting from expired objects, and then treats arriving objects as the continuations of the previous window. Only the expired objects and newly arriving objects need to be processed.

Specifically, the input to the algorithm includes the set of expired objects, $\mathcal{E}$, the set of arriving objects, $\mathcal{A}$, and the candidate set of maximal intervals, $\mathcal{CM}$, which is the output from the previous window. The core steps of the algorithm involve processing each maximal interval $MaxI$ in $\mathcal{CM}$. First, if all objects in $MaxI$ are expired, the interval is removed from the candidate set, and its associated skip pointers are set to NULL, indicating that it is no

Algorithm 7: Incremental Maintenance for Ruzzo-Tompa Algorithm

Input : The expired objects $\mathcal{E}$, the arriving objects $\mathcal{A}$, the candidate set of maximal intervals $\mathcal{CM}$.
Output : Updated $\mathcal{CM}$ after removing expired objects.

for *traverse* $MaxI \in \mathcal{CM}$ *in identified order* **do**
  if $MaxI \cap \mathcal{E} = MaxI$ **then** // all objects of $MaxI$ expire
    Set the skip pointers pointing to $MaxI$ as $NULL$;
    $\mathcal{CM}$.remove($MaxI$);
  else if $MaxI \cap \mathcal{E} \neq \emptyset$ **then** // $MaxI$ includes expired objects
    Recompute new_MaxIs within unexpired objects of $MaxI$ via Algorithm 6;
    $\mathcal{CM}$.add(new_MaxIs);
    Readjust the skip pointers pointing to unexpired objects of $MaxI$;
  else // $MaxI$ does not include expired objects
    break;
Invoke the function Ruzzo-Tompa($\mathcal{CM}$, $\mathcal{A}$)// Algorithm 6;

longer part of the computation. Second, if $MaxI$ includes some expired objects, the algorithm recomputes the maximal intervals for the unexpired objects in $MaxI$ using Algorithm 6. The new maximal intervals are added to the candidate set $\mathcal{CM}$, and the skip pointers pointing to the unexpired parts of $MaxI$ are readjusted. Finally, if $MaxI$ contains no expired objects, the algorithm scans unaffected objects, and they can avoid being checked.

After processing all expired objects, the algorithm invokes the original Ruzzo-Tompa algorithm (inputting the new data set $\mathcal{A}$) to compute the maximal intervals for the new window. The presentation of Ruzzo-Tompa($\mathcal{CM}$, $\mathcal{A}$) means that Algorithm 6 treats arriving objects $\mathcal{A}$ as the continuations of the previous window $\mathcal{CM}$. Specifically, the new objects may merge with the maximal intervals in $\mathcal{CM}$, so we check this mergeability, but we have no necessity to re-scan objects in identified maximal intervals.

B TOURNAMENT-TREE ALGORITHM AND ITS STREAMING VERSION

In this section, we introduce our competing algorithm, the Tournament-Tree algorithm, and its Streaming version.

B.1 Tournament-Tree algorithm (static version)

The Tournament-Tree algorithm (static version) was first proposed in the work [7]. This algorithm directly finds the top-k maximal intervals without comparison, depending on an index called the Tournament Tree. It mainly involves two steps: first building a Tournament tree and then finding the top-k maximal intervals based on this tree. We first introduce the building procedure, outlined in Algorithm 8.

Each node in the tree represents a subsequence of the input sequence and contains the following attributes:

- $(from, to)$: the range of elements covered by this node.
- L: the minimum prefix sum within the coverage, i.e., the minimum prefix sum of the subsequence up to but not including the leftmost element of the range. This is also referred to as the "least".
- the maximum prefix sum within the coverage, i.e., the maximum prefix sum up to the rightmost element of the range. This is also referred to as the "greatest".

Algorithm 8: Build Tournament Tree

Input: Objects $o[f \cdots t]$
Output: Tournament tree

Procedure buildtree($node, f, t$):
  $(from, to) \leftarrow (f, t)$;
  if $from = to$ **then**
    $(L, M, G) \leftarrow (s[f-1], o[f], s[f])$ // node is a leaf;
  else // node is an internal node
    create two children $left$ and $right$;
    buildtree($left, f, \frac{f+t-1}{2}$)// build left subtree;
    buildtree($right, \frac{f+t+1}{2}, t$)// build right subtree;
    $L \leftarrow \min\{left.L, right.L\}$;
    $R \leftarrow \max\{left.G, right.G\}$;
    $M \leftarrow \max\{M_{left}, M_{center}, M_{right}\}$ where $M_{center} \leftarrow right.G - left.L$;

- the maximum sum found within the coverage of the node. This attribute is crucial as it helps compute the maximum sum interval for the given input sequence.

The tournament tree is constructed recursively. At each step, the sequence is divided into two parts: a left subsequence and a right subsequence. The left child node represents the left subsequence, and the right child node represents the right subsequence. Once the left and right children are computed, the node attributes are updated by combining the values from the left and right children. Specifically, L is computed as the minimum prefix sum between the left and right children; G is computed as the maximum prefix sum between the left and right children; M is computed as the maximum value among the left child, the right child, and the sum of the subsequence in between (referred to as M_{center}).

The construction of the tree follows a divide-and-conquer approach, where the recursive construction of left and right subtrees continues until each subsequence corresponds to a single element of the input sequence. Once the tree is built, the root node represents the entire sequence, and the maximum sum subsequence for the entire sequence can be found by accessing the root node.

To find the top-k maximal intervals, we utilize the tournament tree structure. The first step is to compute the top-1 maximal interval, which corresponds to the root of the tournament tree. The root node, as shown in Algorithm 8, contains the maximum sum found in the entire range of the sequence. After identifying the top-1 maximal interval, we treat this interval's range as a "hole," and mark the nodes that cover this range as containing a hole. These nodes are then prevented from being considered for future computations.

Once the top-1 maximal interval is determined, the next step is to find the second maximal interval, ensuring that it does not overlap with the range of the top-1 interval. The corresponding range of the top-1 maximal interval is marked as "hole" in the tournament tree. Specifically, nodes covering this range are marked as invalid, and their maximum values are set to negative infinity ($-\infty$). Then, we update the tree to exclude the range covered by the top-1 interval, and then find the next maximal interval, which is now computed within the remaining valid range. This process is repeated iteratively for the top-k maximal intervals.

B.2 Streaming Tournament-Tree algorithm

We develop an incremental maintenance algorithm for the Tournament-Tree algorithm, detailed in Algorithm 9, to enable it adapt to the streaming context and avoid rebuilding the Tournament tree from scratch as the query window slides.

Algorithm 9: Incremental Maintenance for Tournament-Tree Algorithm

Input : Tournament Tree T, expired objects $\mathcal{E}$, and arriving objects $\mathcal{A}$.
Output : Updated Tournament Tree T.

```
for each expired object e_o ∈ ℰ do
    Delete e_o and and the corresponding subtrees from T;
for traverse T do
    Update "holes" in the Tournament tree: Set affected nodes as normal and
      recalculate their values;
for each new object in o ∈ 𝒜 do
    T.add (o) via Algorithm 8;
return Updated Tournament Tree T.
```

This algorithm operates in three key stages: removing expired objects and the corresponding subtrees, updating “holes” as normal for the new window, and adding new objects to the tree.

First, expired objects are processed by deleting them and their corresponding subtrees from the Tournament Tree. This ensures that only the data corresponding to the current window is retained in the tree, maintaining its relevance and correctness.

Next, after removing expired objects, we update the "holes" in the Tournament Tree. A hole denotes a node that was previously marked invalid but must be reactivated as the window slides. In this stage, affected nodes are set to their normal state, and their values are recalculated to reflect the changes in the tree structure.

Finally, newly arrived objects are incorporated into the tree. This is done by utilizing the Tournament-Tree construction procedure, which builds new subtrees for each new object, ensuring that the tree remains valid and accurate with respect to the current window’s data.

After performing these three operations, the algorithm outputs the updated Tournament Tree, which now reflects the most current data, and top-k maximal intervals for the current window can be retrieved.